Instituto Superior de Engenharia de Lisboa (ISEL)
Departamento de Engenharia Eletrónica e de Telecomunicações e Computadores (DEETC)

LEIM
Licenciatura em Engenharia Informática e Multimédia
Unidade Curricular de Projeto

Again - Jogo Multiplataforma Multiplayer



Fábio Pestana 50756

João Ramos 50730

Orientador

Professor Hugo Cordeiro

junho, 2025

Resumo

Este projeto consiste no desenvolvimento de um jogo **multiplataforma multiplayer**, do tipo **Roguelike**, utilizando o motor de jogo **Unity3D**. O objetivo do jogo é que os jogadores sobrevivam durante o maior número possível de níveis, enfrentando diferentes tipos de inimigos, incluindo inimigos simples e *Bosses*. Após perderem, os jogadores regressam ao *lobby*, onde podem utilizar os recursos adquiridos para melhorar as suas personagens e assim prolongar a sua sobrevivência em sessões futuras. O modo *singleplayer* também é suportado, com o progresso armazenado localmente.

A principal motivação para a realização deste projeto foi aprofundar os conhecimentos sobre *Unity*, desenvolvimento *multiplayer* e criação de jogos multiplataforma, assim como ganhar experiência na gestão e planeamento de um projeto de um jogo completo.

Entre as funcionalidades principais implementadas destaca-se a capacidade de jogadores em diferentes plataformas jogarem em conjunto, através de *lobbies* públicos ou privados.

Os principais contributos incluem a construção de quatro mapas utilizando *assets* disponíveis na *Unity Asset Store*, a implementação de inimigos e três classes jogáveis (guerreiro, mago e clérigo), com animações disponíveis no site *Mixamo*, e efeitos visuais presentes na *Unity Asset Store* como também desenvolvidos pelos autores. O comportamento das habilidades foi programado de raiz, assim como a sincronização *multiplayer*, que utiliza bibliotecas de suporte do *Unity*.

Os testes realizados demonstraram uma boa estabilidade, sincronização e baixa latência em rede local.

O desenvolvimento deste projeto permitiu uma melhoria significativa das competências técnicas em *Unity* e no desenvolvimento de funcionalidades *multiplayer*. Ao longo do trabalho foram também identificadas oportunidades de melhoria no planeamento e organização. Como trabalho futuro, seria interessante adicionar novas classes, maior variedade de inimigos e explorar a geração procedimental de mapas, de forma a enriquecer a experiência de jogo.

Abstract

This project consists of the development of a cross-platform multiplayer game of the Roguelike genre, using the Unity3D game engine. The goal of the game is for players to survive through as many levels as possible, facing different types of enemies, including simple enemies and Bosses. After losing, players return to the lobby, where they can spend the resources acquired during gameplay to improve their characters and thus extend their survival in future sessions. Single-player mode is also supported, with progress storage locally.

The main motivation for carrying out this project was to deepen our knowledge of the Unity engine, multiplayer development, and cross-platform game creation, as well as to gain experience in managing and planning a complete game project.

Among the main features implemented is the ability for players on different platforms to play together through public or private lobbies.

The main contributions include the construction of four maps using assets available on the Unity Asset Store, the implementation of enemies and three playable classes (warrior, mage, and cleric), with animations from the Mixamo website, and visual effects from the Unity Asset Store and developed by the authors. The behavior of the abilities was programmed from scratch, as was the multiplayer synchronization, which uses Unity's Multiplayer library.

Tests conducted demonstrated good stability, synchronization, and low latency on local networks.

The development of this project led to a significant improvement in technical skills in Unity and in the development of multiplayer functionalities. Throughout the project, opportunities for improvement in planning and organization were also identified. As future work, it would be interesting to add new classes, a greater variety of enemies, and explore procedural map generation to further enrich the gaming experience.

Índice

Resumo **i**

Abstract **iii**

Índice **v**

Lista de Tabelas **vii**

Lista de Figuras **ix**

1 Introdução **1**

2 Trabalho Relacionado **3**

3 Modelo Proposto **7**

3.1 Requisitos 7

3.2 Fundamentos 9

3.2.1 Fundamentos do Género Roguelike 9

3.2.2 Fundamentos do Desenvolvimento Multiplayer 10

3.2.3 Tecnologias Utilizadas 11

3.3 Abordagem 13

3.3.1 Personagens Jogáveis 13

3.3.2 Sistema de Inteligência Artificial 13

3.3.3 Sincronização dos jogadores na rede 14

3.3.4 Persistência de Dados 14

3.3.5 Sistema de Lobbies e Conectividade 15

3.3.6 Gestão da Sessão e Progressão do Jogo 15

4 Implementação do Modelo **17**
4.1 Sistema Multiplayer 17
4.1.1 Netcode for GameObjects 17
4.1.2 Unity Services: Lobby 19
4.1.3 Unity Services: Relay 21
4.2 Mapa 22
4.3 Personagens e Classes 25
4.3.1 Knight 26
4.3.2 Mage 28
4.3.3 Cleric 30
4.4 Inimigos 32
4.5 SinglePlayer 36
4.6 UI 37
4.6.1 Visualização de Vida e Estado do Jogador 38
4.6.2 Controlo de Habilidades e Plataforma 38
4.6.3 Atualização de Imagens e Progresso de Habilidades . . 38
4.6.4 Menus e Transições 39
4.6.5 Botão de Guardar Jogo 39
4.6.6 Interligação com o Menu Inicial 40

5 Validação e Testes **43**

6 Conclusões e Trabalho Futuro **45**

A Design das salas **47**

B Questionário de Satisfação **53**

Bibliografia **61**

Lista de Tabelas

Lista de Figuras

2.1 Sessão de jogo de Soul Knight. 3
2.2 Criação de Lobby no jogo Lethal Company. 4

3.1 Diagrama de casos de utilização principais do sistema. 9

4.1 Exemplo de um Lobby criado. 21
4.2 Zona do Nexus presente no Menu principal. 23
4.3 Sala com os perigos do ambiente. 24
4.4 Classe jogável Knight. 27
4.5 Classe jogável Mage. 29
4.6 Classe jogável Cleric. 31
4.7 Animator dos inimigos. 33
4.8 Sub-State Machine Patrol do inimigo. 33
4.9 Sub-State Machine Attack do inimigo. 34
4.10 Inimigos base - Esqueletos. 35
4.11 Boss Minotauro. 36
4.12 UI durante o jogo. 39
4.13 UI do menu de opções do jogo. 40
4.14 Menu principal do jogo. 41

Capítulo 1

Introdução

Este projeto consiste no desenvolvimento de um jogo **multiplataforma multiplayer**, do tipo **Roguelike**, utilizando o motor de jogo Unity3D. O objetivo principal foi aprofundar conhecimentos sobre o desenvolvimento de funcionalidades *multiplayer* em *Unity* e compreender como estruturar um projeto de um jogo completo, com suporte multiplataforma.

A principal inspiração para este trabalho foi o jogo **Soul Knight**, um jogo popular de ação *Roguelike* disponível para plataformas móveis. Jogos *Roguelike* caracterizam-se por apresentarem níveis gerados de forma procedimental, nível de dificuldade adaptável que depende tanto da força da personagem do jogador como do seu conhecimento sobre o jogo, morte permanente por tentativa e **forte valor de replay**. Em *Soul Knight*, os jogadores exploram masmorras, derrotam inimigos e recolhem itens para fortalecer as suas personagens. O projeto procura recriar um conceito semelhante, mas com um foco em *multiplayer* e em suporte multiplataforma, o que permite que jogadores em diferentes dispositivos joguem em conjunto.

O jogo distingue-se pelo facto de suportar **multiplayer cross-platform**, isto é, permite que jogadores em dispositivos com sistemas operativos diferentes (Windows e Android) possam jogar na mesma sessão. Este tipo de funcionalidade é incomum em jogos *Roguelike* e apresenta desafios técnicos relevantes a nível da sincronização.

Entre as principais contribuições do trabalho destacam-se a criação de **mapas originais** a partir de *assets* modulares (estruturas, chão, paredes e objetos decorativos), a implementação de **comportamentos personalizados** para inimigos e o desenvolvimento de **três classes jogáveis** — Guer-

reiro, Mago e Clérigo — cada uma com um ataque básico e três habilidades especiais. Foi também criado um *particle system* original para uma das habilidades. Uma funcionalidade adicional que diferencia o jogo é o sistema de movimento no Windows, onde a personagem do jogador está sempre orientada para a posição do cursor do rato, com animações adaptadas em tempo real à direção de visão, um detalhe pouco comum em jogos deste tipo.

O processo de desenvolvimento seguiu um planeamento estruturado com metas semanais. Iniciou-se com a criação de **builds** simples para testar as capacidades *multiplayer* e a viabilidade do *crossplay*. Posteriormente, foi elaborado um documento **GDD** (*Game Design Document*), que detalha a visão geral, mecânicas de jogo, personagens, controlos, câmara, universo do jogo, inimigos, interface e cronograma de desenvolvimento. O sistema de **lobbies** foi implementado com o apoio da biblioteca *Unity Lobby*. Foi criado um mapa para o *lobby* e outros quatro mapas de jogo, cada um composto por diversas salas (oito tipos de salas diferentes). Em seguida, foram desenvolvidas as três classes jogáveis, os inimigos que os jogadores enfrentam e o sistema de progressão e *upgrades* das personagens. Por fim, foi implementado um sistema de gravação de progresso, permitindo guardar e recuperar sessões de jogo.

Durante a fase de testes, não foram identificados problemas de desempenho nos dispositivos utilizados, com a aplicação a demonstrar boa estabilidade e sincronização em redes locais.

De uma forma geral, o projeto atingiu os objetivos propostos, visto que permitiu aos alunos aprofundarem significativamente os seus conhecimentos sobre *Unity*, desenvolvimento *multiplayer* e gestão de projetos de jogos. A experiência revelou também oportunidades de melhoria na organização e planeamento de projetos futuros. Em termos de evolução do jogo, existem várias possibilidades de enriquece-lo, nomeadamente através da introdução de maior complexidade e diversidade de estilos de jogo através de novas classes, itens e inimigos, e da implementação de um sistema de geração procedimental de mapas o que aproximaria mais este jogo a outros jogos do seu género.

Este relatório está organizado em seis capítulos: a Introdução; o Trabalho Relacionado; o Modelo Proposto (Requisitos, Fundamentos e Abordagem); a Implementação do Modelo; a Validação e Testes; e, por fim, as Conclusões e propostas para Trabalho Futuro.

Capítulo 2

Trabalho Relacionado

Este projeto enquadra-se no contexto dos jogos *Roguelike* com suporte para *multiplayer cross-platform*, que combina um género de jogo com um elevado valor de *replay* com os desafios técnicos da sincronização de ambientes dinâmicos entre jogadores em diferentes plataformas.



Figura 2.1: Sessão de jogo de Soul Knight.

O desenvolvimento deste projeto foi principalmente inspirado pelo jogo *Soul Knight* [ChillyRoom, 2017], um *Roguelike* de ação com níveis gerados de forma procedimental e uma grande variedade de personagens e armas. A estrutura geral da *gameplay* em *Soul Knight*, baseada em ciclos de progressão e combate em múltiplas salas, foi uma referência fundamental para a conceção do *loop* de *gameplay* central do jogo, no qual os jogadores enfrentam

ondas de inimigos em ambientes variados e progridem ao aprimorar as suas personagens através de recursos adquiridos nos abates dos monstros.

A nível estético e interação, elementos do jogo *Hades* [Supergiant, 2020], como a atenção ao detalhe nas animações, foram uma inspiração importante. No entanto, para a componente de controlo e orientação da personagem, a principal referência foi *Nuclear Throne* [Vlambeer, 2015], um *Roguelike top-down* no qual a personagem está permanentemente orientada para o cursor do rato, com animações dinâmicas ajustadas em tempo real. Esta abordagem foi particularmente relevante para a implementação do sistema de movimento e controlo, que procura proporcionar uma sensação de fluidez e usabilidade intuitiva ao jogador.

Em relação à componente *multiplayer*, estudámos o sistema de lobbies e gravação de progresso do jogo *Lethal Company* [Zeekerss, 2023], que oferece uma experiência baseada em sessões persistentes e partilhadas entre jogadores. Este modelo serviu de inspiração para a implementação do próprio sistema de lobbies, com integração de um sistema de gravação de progresso associado a cada sessão de jogo.



Figura 2.2: Criação de Lobby no jogo Lethal Company.

Durante a fase de planeamento, recorreu-se às boas práticas de desenvolvimento de jogos, elaborando um *Game Design Document* (GDD), que serviu como guia estruturante ao longo do projeto. Para a construção deste documento, consultámos recursos especializados como o repositório de exem-

plos de GDD disponibilizado em [GameScrye, 2024], que ajudaram a definir componentes essenciais como a visão geral do jogo, mecânicas, personagens, inimigos, interface, controlos e cronograma.

Adicionalmente, para garantir uma compreensão sólida das ferramentas multiplayer utilizadas, apoiámo-nos nos manuais oficiais do *Unity Multiplayer Solutions* [Unity Technologies, 2024b] e nas diretrizes para integração de *Relay* com *Netcode for GameObjects* [Unity Technologies, 2024e], que forneceram suporte técnico essencial para a correta implementação das funcionalidades de rede.

Os aspetos diferenciadores do projeto, no contexto dos jogos *Roguelike*, incluem o suporte para *multiplayer cross-platform*, um sistema de controlo e orientação da personagem pelo cursor do rato com animações adaptativas, e um sistema de gravação de progresso associado a *lobbies*, inspirado em abordagens modernas como a do *Lethal Company*. Cada uma destas componentes foi desenvolvida com base em conceitos e tecnologias cuidadosamente analisados ao longo do projeto e adaptada de acordo com as necessidades específicas da nossa aplicação.

Capítulo 3

Modelo Proposto

Este capítulo apresenta o modelo proposto para o desenvolvimento do jogo multiplataforma *multiplayer* do tipo *Roguelike*. O percurso inicia com a definição dos requisitos, segue com os fundamentos teóricos e tecnológicos que sustentam o projeto e culmina com a descrição da abordagem adotada.

3.1 Requisitos

Para orientar o desenvolvimento do projeto, foi realizada uma análise de requisitos que resultou na identificação de funcionalidades essenciais, requisitos não funcionais e casos de utilização prioritários. A metodologia seguida teve como base uma abordagem iterativa e incremental, em que funcionalidades foram discutidas, planeadas e validadas ao longo do progresso do desenvolvimento.

Requisitos Funcionais

Os principais requisitos funcionais são:

- Permitir a criação de sessões de jogo (*lobbies*), públicas ou privadas.
- Suportar *multiplayer cross-platform* entre utilizadores Windows e Android.
- Permitir progressão do jogador com a recolha e gasto de recursos adquiridos a derrotar inimigos.

- Disponibilizar três classes jogáveis distintas: Guerreiro, Mago e Clérigo.
- Implementar inimigos escaláveis em dificuldade.
- Guardar o progresso do jogador localmente (modo *singleplayer*) ou por sessão (modo *multiplayer*).

Requisitos Não Funcionais

- O sistema deve garantir baixa latência e sincronização no modo *multiplayer*.
- A interface deve ser intuitiva e adaptada a diferentes plataformas.
- O jogo deve operar com um bom desempenho, isto é, sem quedas de *frames* significativas.

Casos de Utilização

Os principais casos de utilização do sistema estão representados no Diagrama da Figura 3.1. Este diagrama evidencia as ações que os utilizadores (jogadores) podem realizar, tanto no modo *singleplayer* como no modo *multiplayer*.

No início da interação, o jogador pode optar por iniciar uma sessão em modo *singleplayer*, ou por criar ou juntar-se a uma *sala multiplayer*. No caso da criação de sala, o jogador que a cria assume automaticamente o papel de *Host*. Antes de iniciar o jogo, o utilizador pode também aceder às *configurações do jogo*, que incluem opções de controle do volume e brilho do jogo.

Estas ações comunicam com os serviços da Unity, nomeadamente *Unity Lobby* e *Unity Relay*, responsáveis pela gestão das sessões e encaminhamento de dados entre os jogadores.

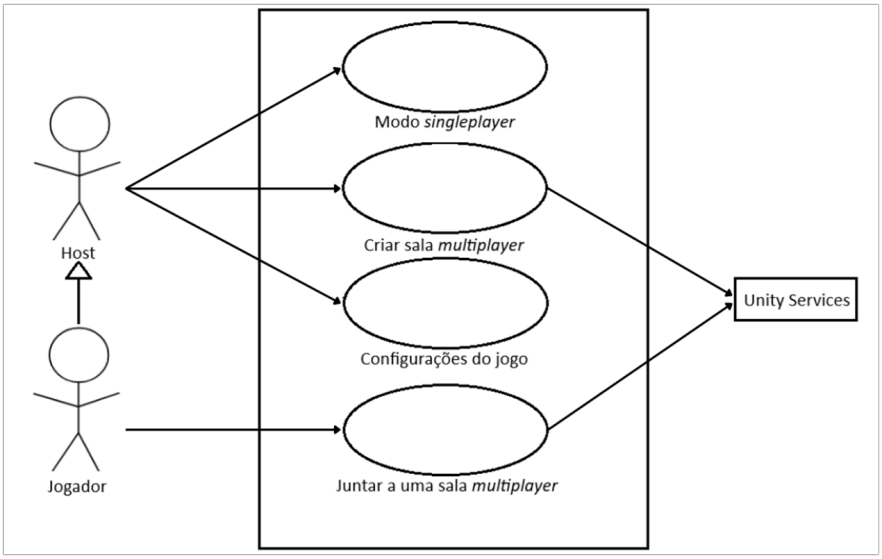


Figura 3.1: Diagrama de casos de utilização principais do sistema.

3.2 Fundamentos

Os fundamentos do projeto assentam nas áreas de jogos do género *Roguelike* e as suas características estruturantes e os princípios teóricos associados ao desenvolvimento de jogos *multiplayer* sincronizados.

3.2.1 Fundamentos do Género Roguelike

O género *Roguelike* tem como principais características a geração procedimental de níveis, a morte permanente e um elevado grau de valor de *replay*. Ao contrário de jogos com estrutura linear ou narrativa rígida, um *Roguelike* proporciona ao jogador uma experiência diferente a cada nova tentativa, promovendo assim a repetição e uma progressão do sistema de jogo.

No projeto, estas ideias foram aplicadas através da organização dos níveis em salas distintas, com conteúdos variáveis, que incluem tipos de inimigos, obstáculos e configurações espaciais diferentes. Visto que o projeto não enquadra o sistema de geração procedimental dos níveis, o grupo tomou a decisão de desenvolver salas diferentes, e a partir delas desenvolver 4 (quatro) níveis de jogo distintos ao organizar estas salas de maneiras diferentes.

A ausência de *checkpoints* e a exigência de recomeçar do início após a

derrota conferem ao jogador uma motivação adicional para evoluir a sua personagem, explorar novas estratégias e dominar os sistemas de combate, para além da utilização do sistema de progressão das personagens onde os jogadores podem desbloquear novas habilidades para as mesmas. O ciclo de jogar-perder-aprender-jogar é central à experiência proposta.

Além disso, o modelo incorpora mecânicas inspiradas em jogos *Roguelike* modernos que combinam a estrutura clássica com elementos de combate em tempo real e desbloqueio de novas habilidades.

3.2.2 Fundamentos do Desenvolvimento Multiplayer

A componente *multiplayer* do projeto exigiu a consideração de vários conceitos teóricos fundamentais para garantir uma experiência de jogo sincronizada e responsiva entre jogadores em diferentes plataformas e redes.

Um dos conceitos centrais é a noção de autoridade. Este termo refere-se à responsabilidade sobre a decisão final dos eventos que ocorrem no jogo — por exemplo, quem valida um ataque, um movimento ou uma colisão. O modelo adotado foi o de autoridade no *host*, uma abordagem cliente-servidor em que um dos jogadores assume o papel de anfitrião. Este *host* é responsável por manter o estado global do jogo e por validar todas as interações entre jogadores e ambiente. Esta abordagem simplifica o modelo de sincronização, dado que reduz os conflitos entre múltiplas fontes de autoridade e centraliza a lógica crítica numa única instância.

Outro conceito essencial é a sincronização de estados. Numa aplicação *multiplayer* em tempo real, é necessário garantir que todos os jogadores vejam uma versão coerente e atualizada do estado do jogo. Essa sincronização envolve a replicação de variáveis como a posição de objetos, estado de animações, eventos de combate e interações com o ambiente. No projeto, este processo é facilitado pela biblioteca *Netcode for GameObjects*, que fornece mecanismos internos para partilha automática de variáveis de rede (`NetworkVariables`), envio de chamadas de eventos remotos (`ServerRpc` e `ClientRpc`) e controlo da propriedade de objetos de rede. O modelo tira partido das funcionalidades básicas fornecidas pela biblioteca, o que permite que o estado dos objetos controlados pelo jogador e os eventos principais do jogo sejam replicados automaticamente entre o *host* e os clientes, sem necessidade de desenvolver um sistema de rede personalizado.

3.2.3 Tecnologias Utilizadas

Do ponto de vista tecnológico, o projeto foi desenvolvido com o motor Unity3D, na sua versão 6 (seis), que oferece melhorias significativas em termos de desempenho e novas funcionalidades como *light probe volumes*, permitindo otimizar a iluminação e manter a *performance* em diferentes plataformas. Para a componente de rede e conectividade, um dos principais desafios do projeto, utilizámos as soluções oficiais da Unity: *Unity Relay* [Unity Technologies, 2024d] e *Netcode for GameObjects* [Unity Technologies, 2024c], que forneceram as bases técnicas para a comunicação em tempo real entre clientes. Para a implementação do sistema de lobbies, foi utilizada a biblioteca *Unity Lobby* [Unity Technologies, 2024a], cuja integração permitiu gerir sessões de jogo de forma eficaz e escalável.

Unity é o motor de jogo escolhido para o desenvolvimento do projeto. É utilizado por vários desenvolvedores independentes visto que é gratuito e contém diversos recursos disponíveis na sua *Asset Store* que atua como uma loja *online* que permite aos desenvolvedores adquirir *Assets* elaborados por outros desenvolvedores, de modo a facilitar o desenvolvimento dos projetos realizados em Unity. Neste projeto, foram utilizados diversos *Assets* em múltiplos aspetos, como animações, modelos e elementos do mapa, todos eles transferidos de forma gratuita. Outro motivo para a escolha do *Unity* como motor de jogo foi a experiência adquirida ao longo do curso, tanto em termos da utilização do motor de jogo *Unity* como também na linguagem de programação *C#* utilizada no mesmo.

Unity Lobby fornece um serviço baseado na nuvem que permite a criação e descoberta de sessões de jogo. Quando um jogador cria um *lobby*, o Unity *Lobby* gera um identificador único (ID) para a sessão e regista-a nos servidores da Unity, juntamente com os metadados associados (por exemplo, nome da sala, número máximo de jogadores e estado atual). Outros jogadores podem listar *lobbies* públicos disponíveis ou juntar-se a um *lobby* específico através desse ID. A biblioteca gere o ciclo de vida do *lobby* (criação, entrada, saída, encerramento), assim como sincroniza automaticamente os dados partilhados entre os membros da sessão antes do início do jogo.

Neste projeto está a ser utilizada a versão gratuita (*free tier*) do Unity

Lobby. Esta versão oferece funcionalidades necessárias, mas com algumas limitações em relação ao plano pago: o número máximo de jogadores por *lobby* está limitado a 10, há um número reduzido de chamadas por minuto para operações de leitura e escrita nos metadados do *lobby*, e a retenção de dados da sessão é limitada a 14 dias. Apesar destas limitações, a versão gratuita é suficiente para os objetivos do projeto, visto que permite criar *lobbies* públicos e privados com múltiplos jogadores.

Unity Relay é um serviço que resolve o problema da conectividade entre jogadores localizados em redes privadas (como NATs ou *firewalls*) que impedem ligações diretas. O *Relay* atua como um servidor intermediário que recebe e reenvia os dados entre clientes. Quando um *lobby* é iniciado, é gerado um *Relay Join Code* que identifica a rota dos dados. Os jogadores enviam e recebem mensagens UDP através do *Relay*, que garante que os pacotes chegam aos destinatários corretos mesmo que estes não tenham IPs públicos acessíveis. Este modelo elimina a necessidade de configurar portas no *router* ou usar soluções como *port forwarding*, o que torna o sistema acessível a qualquer jogador com ligação à Internet.

Neste projeto está a ser utilizada a versão gratuita (*free tier*) do Unity *Relay*, que permite até 50 CCU (*concurrent users*) e uma largura de banda mensal de 3 GB por mês. As versões pagas oferecem maior escalabilidade, mais CCUs simultâneos, maior volume de dados mensais e acesso a suporte técnico prioritário. No entanto, para o contexto deste projeto — que visa suportar *lobbies* pequenos de até 3 jogadores e partidas de curta duração a versão gratuita é mais que o suficiente.

Netcode for GameObjects (NGO) , como já referido anteriormente, é a biblioteca responsável pela sincronização real do estado do jogo entre o *host* e os clientes. Funciona sobre o *transport layer* (como o *Relay*), e fornece abstrações para marcar variáveis como sincronizáveis (`NetworkVariable`), invocar métodos remotos (`ServerRpc` e `ClientRpc`) e identificar instâncias de objetos com IDs únicos de rede. Esta estrutura permite que os objetos da Unity (`GameObjects`) sejam "visíveis"na rede, ou seja, o que acontece numa máquina deve acontecer noutras automaticamente.

Com estas três bibliotecas a atuar em conjunto foi possível estruturar

uma arquitetura *multiplayer* suficiente para o projeto.

3.3 Abordagem

O desenvolvimento deste projeto baseou-se numa abordagem que combina um sistema de rede, inteligência artificial e um modelo para persistência de dados. A arquitetura foi concebida para suportar interações entre os jogadores e o sistema de jogo, mantendo ao mesmo tempo a *performance* e responsividade do jogo *multiplayer*.

3.3.1 Personagens Jogáveis

O núcleo do sistema encontra-se na classe base das personagens jogáveis `CharacterBase`, esta classe define os atributos e comportamentos comuns entre todas as classes jogáveis (como a troca entre níveis de jogo, evento de morte e de voltar à vida do jogador, entre outros). Esta abordagem orientada a objetos permite que cada classe que herda da anterior (`Knight`, `Mage` e `Cleric`) implementa as suas características únicas através de polimorfismo. Um exemplo, é na classe `Cleric` que tem uma habilidade para curar todos os jogadores dentro de uma área, esta habilidade é implementada através de um `ServerRPC`, que verifica numa área se existe jogadores e se eles existirem chama o seu método `Heal` (do `CharacterBase`) que o cura.

3.3.2 Sistema de Inteligência Artificial

O sistema de inteligência artificial para os inimigos consiste na utilização de máquinas de estados finitos, com os comportamentos: Patrulha (consiste no inimigo andar por uma sala em pontos aleatórios), Perseguição (segue o jogador mais próximo) e Ataque (inimigo realiza rotações segundo o jogador e ataca-o). Os inimigos utilizam o algoritmo de *pathfinding* (o algoritmo de procura A*) para navegarem pelo ambiente de forma eficiente e também utilizam um sistema de deteção por `Raycast`, para permitir aos inimigos terem "visão". Os inimigos simples foram implementados na classe base `Skeleton`, que define os atributos e comportamentos comuns entre todos os inimigos, estas classes apenas são executadas no servidor, pois os inimigos não são controlados pelos clientes, logo só o cliente que está a executar o

servidor é que tem o peso da execução destes códigos. Para a implementação de um inimigo mais forte, um *Boss*, criou-se a classe filha de `Skeleton`, `Boss`, que implementa um inimigo que diferencia-se dos anteriores por ter duas fases. Quando atinge metade da vida o inimigo fica mais agressivo (diferentes animações de ataques e dá mais dano aos jogadores).

3.3.3 Sincronização dos jogadores na rede

Para realizar a sincronização entre os jogadores na rede, optou-se por um modelo que combina *client-authoritative movement* e animações com *server-authoritative* para ações mais críticas de interação entre os componentes do jogo (cálculo e aplicação de danos a inimigos e jogadores, aplicação de efeitos, ganhar moedas, gastar moedas para aumentar o nível do jogador, entre outros). Componentes como `NetworkTransform` e `NetworkAnimator`, foram configurados com autoridade no cliente, o que permite que a movimentação e as animações sejam responsivas localmente. Já o servidor mantém controlo total sobre o estado do jogo, garantindo consistência e segurança nas interações importantes. Para tal, utiliza-se `NetworkVariables` para sincronizar os estados essenciais, como vida, estado de ataque e moedas, enquanto `ServerRPCs` e `ClientRPCs` lidam com eventos do jogo, como o uso de habilidades. Um exemplo concreto deste sistema ocorre no ataque básico do `Mage`, que cria um projétil sincronizado em rede: o cliente inicia a animação localmente, mas a instanciação do projétil e o cálculo de dano são realizados no servidor por meio de um `ServerRPC`.

3.3.4 Persistência de Dados

A persistência de dados foi implementada através da classe estática `SaveSystem`. Quando um jogador entra no jogo pela primeira vez, é-lhe pedido um nome, que é guardado juntamente com um identificador único global (GUID: *Globally Unique Identifier*). Este identificador é gerado automaticamente através da classe `System.Guid`, e tem como objetivo garantir que cada jogador tem um ID único que o distingue de todos os outros, mesmo que múltiplos jogadores tenham o mesmo nome.

A utilização de GUIDs é uma excelente escolha neste contexto, pois oferece várias vantagens. Em primeiro lugar, garante unicidade: a probabilidade

de gerar dois GUIDs iguais é praticamente nula, o que é essencial num sistema *multiplayer* onde múltiplos jogadores podem partilhar nomes semelhantes ou até idênticos. Em segundo lugar, os GUIDs permitem uma identificação segura e confiável dos jogadores nas sessões guardadas, facilitando a associação correta dos dados persistidos a cada jogador. Isto é especialmente útil ao carregar sessões antigas, pois evita trocas indevidas de informações.

As sessões de jogo são representadas pela classe `GameSessionData`, que contém dados como o nome da sessão, se é *multiplayer* e a lista de jogadores (`PlayerSessionData`). Cada jogador é identificado pelo seu GUID, e os seus dados: nível, número de moedas e classe jogável. Estes são guardados de forma persistente, para tal são serializados em ficheiros binários e armazenados localmente (nome e ID do utilizador em cada dispositivo do utilizador e os dados das sessões são guardados no utilizador que está a fazer *Host* do jogo), permitindo guardar e carregar sessões de forma segura e eficiente. O sistema também permite atualizar sessões existentes, distinguir entre sessões *singleplayer* e *multiplayer*, e aceder a sessões específicas através do seu nome.

3.3.5 Sistema de Lobbies e Conectividade

Para os jogadores conseguirem jogar em conjunto, desenvolveu-se um sistema de rede que utiliza-se os serviços do Unity, o sistema de *Lobbies*, permite a criação e junção de partidas através de códigos (para entrar num *lobby* privado) ou através de uma lista de *lobbies* públicos. Implementamos um mecanismo de *heartbeat* para manter os *lobbies* ativos (pois, por padrão, *lobbies* sem atividade são encerrados pelo serviço do Unity) e um sistema que verifica quando todos os jogadores estão preparados para iniciar a partida. Durante a transição do *lobby* para o jogo propriamente dito, utilizamos o serviço Unity *Relay* para estabelecer conexões P2P facilitadas.

3.3.6 Gestão da Sessão e Progressão do Jogo

Dentro de uma sessão de jogo, a progressão através das *dungeons* é gerenciada pelo `GameManager`, que controla a escolha entre as diferentes *scenes* (cada uma com um mapa diferente) segundo o nível do jogo e a transição entre elas. Quando os jogadores derrotam pelo menos oito inimigos ou um *Boss* em um nível, o sistema verifica se os jogadores estão preparados para ir

para o próximo nível e, quando todos estão preparados, carrega o próximo mapa. Caso o nível de jogo for o último todos os jogadores voltam para o primeiro mapa (designado por *Nexus*) onde eles podem gastar os recursos que ganharam durante a sessão. Se todos os jogadores estiverem mortos o `GameManager`, revive todas as personagens e envia-os para o primeiro mapa do jogo.

Capítulo 4

Implementação do Modelo

4.1 Sistema Multiplayer

De modo a atingir o objetivo do jogo ser jogado em *Multiplayer Online*, isto é, permitir que vários jogadores em dispositivos e redes diferentes possam jogar juntos numa mesma sessão de jogo, implementámos o seguinte modelo com recurso às tecnologias previamente referidas na secção 3.2.3.

4.1.1 Netcode for GameObjects

A biblioteca **Netcode for GameObjects** é uma biblioteca, disponibilizada pelo *Unity*, que permite abstrair a lógica da camada de rede, visto que, contém múltiplos métodos que facilitam a transmissão de informações relacionadas com *GameObjects* e a cena durante uma sessão de jogo online, para múltiplos jogadores em simultâneo.

Todos os `GameObjects` que devem estar sincronizados entre os múltiplas dispositivos envolvidos na comunicação devem herdar de **NetworkBehaviour** em vez de `MonoBehaviour`, sendo este o `Behaviour` padrão do *Unity* definido em todos os scripts criados por omissão. Alguns exemplos são as personagens jogáveis e os inimigos.

De modo a que seja possível estabelecer uma sessão entre múltiplos jogadores em simultâneo, é necessário adicionar um `GameObject` à cena que possua o componente `Network Manager`, visto que este componente é responsável por toda a gestão da comunicação de rede, incluindo o protocolo a ser utilizado e o tipo de objeto que será instanciado na cena de modo a representar o jogador, no atributo `Player Prefab`.

Após a definição do `Network Manager` na cena, é possível iniciar uma ligação como *Host*, Servidor ou Cliente, ao entrar no Modo *Play* do *Unity* e selecionar uma destas três opções que se encontram exibidas no componente `Network Manager` durante este modo. No âmbito deste projeto, os jogadores serão designados como sendo *Hosts* ou Clientes. Os dispositivos dos *Hosts* funcionarão como **Cliente-Servidor**.

Após esta configuração inicial da cena, é necessário que alguns scripts executem **Remote Procedure Calls** (ou RPCs), estas são chamadas remotas ao servidor ou ao cliente com o principal objetivo de executar determinadas funções.

O objetivo de uma RPC consiste na realização de uma chamada remota para executar um determinado método. Por exemplo, os scripts das classes jogáveis `(Mage, Knight e Cleric)` fazem uso de RPCs para sincronizar habilidades entre os clientes e o *Host* (cliente-servidor). Estas chamadas são essenciais para garantir que ações como ataques, habilidades especiais e efeitos visuais sejam replicados de forma consistente e síncrona em todos os dispositivos conectados. Abaixo, encontra-se um exemplo técnico:

- `MageSkill2ServerRPC`: Quando o jogador usa uma habilidade de área (como um meteoro de gelo), as coordenadas do alvo (`worldPos`) são enviadas ao servidor via `ServerRpc`. O servidor então instancia o efeito e calcula o dano em todos os inimigos dentro do raio, garantindo consistência.

- `KnightUltimateServerRPC`: Instancia um projétil com dano sincronizado, cuja trajetória é calculada no servidor para prevenir discrepâncias entre os clientes.

- `AreaHealServerRpc`: Propaga a cura para todos os jogadores dentro de um raio ao usar `NetworkVariables` para atualizar a vida das personagens (character.Heal()).

- `ReviveAllPlayersServerRpc`: Ressuscita jogadores mortos iterando sobre a lista `GameManager.Instance.players`, demonstra assim o uso de estados globais sincronizados.

4.1.2 Unity Services: Lobby

O **Unity Lobby** é um serviço online fornecido pela `Unity` que permite aos jogos *multiplayer* criarem e gerirem sessões de jogo antes da conexão direta entre jogadores. A sua principal função é servir como uma "sala de espera" onde os jogadores podem-se reunir, configurar as opções da sessão (como nome da sala e privacidade) e aguardar que todos estejam prontos antes de iniciar efetivamente o jogo.

Por trás deste sistema está uma arquitetura baseada em servidores dedicados mantidos pela *Unity*, que armazenam e atualizam o estado dos *lobbies* de forma temporária. Cada *lobby* pode ser público (visível para outros jogadores) ou privado (acessível apenas por convite ou código). Os *lobbies* suportam atualizações em tempo real usando o conceito de *heartbeat* (batimento), um sinal enviado periodicamente pelo *host* para informar que o *lobby* continua ativo. Caso o *heartbeat* não seja recebido por um período de tempo, o *lobby* é automaticamente encerrado, garantindo limpeza de sessões inativas.

A gestão dos *lobbies* também permite o armazenamento de meta-dados personalizados e permite que cada jogador possa ter dados associados ao seu estado individual dentro do *lobby* (como nome ou classe selecionada).

A implementação do sistema de *lobbies* no projeto divide-se em dois componentes principais: o `LobbyManager`, responsável pela lógica de comunicação com os serviços do *Unity*, e o `ManagerUI`, utiliza os métodos do `LobbyManager` para tratar da interface em relação aos *lobbies* e trata da interação com os utilizadores.

O `LobbyManager` tem como função principal: criar, gerir, atualizar e eliminar *lobbies* conforme a interação dos jogadores. Esta classe inicia a sua lógica no método `Start()`, onde os serviços da *Unity* são inicializados com `UnityServices.InitializeAsync()`. De seguida, é feita a autenticação anónima do jogador através do `AuthenticationService.Instance.SignInAnonymouslyAsync()` e ao fazer com que lhe seja atribuído um identificador único temporário, necessário para todas as interações seguintes com os serviços do *Unity*.

Logo após a autenticação, dois métodos importantes são invocados no método `Update()`: `HandleLobbyUpdates()` e `KeepLobbyAlive()`. O primeiro é responsável por atualizar os dados do *lobby* em intervalos regulares

de três segundos, e também verifica se o jogo já começou (quando todos os *players* estão prontos a jogar) para utilizar o *Unity Services Relay*. O segundo método envia o designado *heartbeat* a cada quinze segundos para manter o *lobby* ativo, isto é fundamental, pois apenas o host é responsável por evitar que o *lobby* expire (um *lobby* é desativado automaticamente se passar trinta segundos sem atividade).

A criação de um novo *lobby* é feita pelo método `CreateLobby(string name, bool isPrivate)`. Este método utiliza o `LobbyService.Instance.CreateLobbyAsync()`, onde é definido o nome da sala, o número máximo de jogadores (três jogadores) e o nível de visibilidade (público ou privado). O jogador que cria o *lobby* é automaticamente adicionado como *host*, e os seus dados são registados com propriedades personalizadas (método `GetPlayer()`), como `PLAYER_NAME` (nome do jogador), `PLAYER_CHARACTER` (variável para guardar a classe do jogador escolhida) e `PLAYER_IS_READY` (variável que indica se o jogador está ou não pronto para jogar). Logo após a criação do *lobby*, este é guardado em duas variáveis: `hostlobby`, que representa a versão local do *lobby* criado, serve para indicar que o jogador é o *host*, pois os outros jogadores ao juntarem-se ao *lobby* terão esta variável com valor `null`, e `joinedlobby`, utilizada pelo *host* e clientes para atualizar as informações do *lobby*.

A entrada de um jogador num *lobby* pode ser feita de duas formas, através de `JoinLobby(string code, string name)` ou *JoinLobbyById(string id, string name)*, dependendo se o jogador pretende entrar por código de convite (coloca o código num input *text* da interface) ou diretamente pelo identificador do *lobby* (cada *lobby* público é apresentado na interface através de um botão). Ambas as funções utilizam o mesmo padrão de construção do objeto jogador com `GetPlayer()` e atualizam a variável `joinedlobby`.

O `LobbyManager` também oferece funcionalidades para sair do *lobby* através do método `LeaveLobby()`, que remove o jogador da sessão ativa através do método `LobbyService.Instance.RemovePlayerAsync()`. Quando o *host* pretende iniciar o jogo, invoca-se o método `StartGame()`, que cria uma sessão de *Relay* através do `RelayManager`, este retorna o código de conexão e atualiza o estado do *lobby* com esse código, permitindo assim aos restantes jogadores conectarem-se à sessão de jogo.

Os jogadores também conseguem interagir com o *lobby* ao atualizar o

nome da classe escolhida com `UpdatePlayerCharacter(string nameOfClass)` ou indicar que estão ou não estão prontos para o jogo através do método `ToggleReadyStatus(bool ready)`. Por fim, o método `AreAllPlayersReady()` percorre todos os jogadores no *lobby* e verifica se todos os jogadores estão prontos, retorna `true` se todos tiverem prontos, esta verificação é necessária para permitir o *host* começar o jogo com o método `StartGame()`.

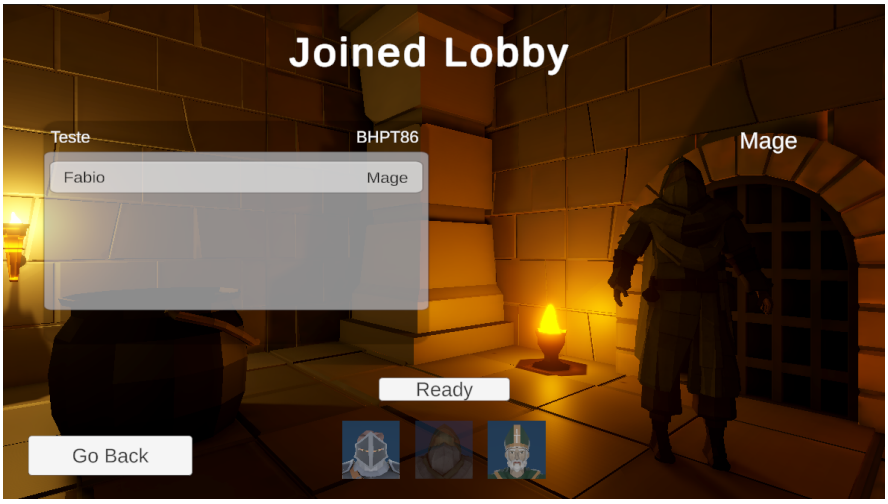


Figura 4.1: Exemplo de um Lobby criado.

4.1.3 Unity Services: Relay

O **Unity Relay** é um serviço de encaminhamento de tráfego de rede que permite a comunicação entre jogadores em jogos *multiplayer* sem a necessidade de um servidor dedicado ou que os jogadores tenham de expor as suas portas de rede (*port forwarding*). Em vez de uma ligação direta *peer-to-peer* tradicional, que pode ser bloqueada por *firewalls* ou NAT (*Network Address Translation*), o *Relay* atua como um intermediário que transmite as mensagens entre os clientes e o *host*.

Na prática, o *Relay* funciona como um servidor de encaminhamento, sem executar qualquer lógica do jogo, apenas retransmite pacotes entre os jogadores conectados.

No projeto, a integração com o *Unity Relay* ocorre em conjunto com o sistema de *Lobby*. Quando todos os jogadores de um *lobby* estão prontos, o *host* utiliza o método `CreateRelay()` da classe `RelayManager` que invoca o

`RelayService.Instance.CreateAllocationAsync()`, que aloca uma sessão para até três jogadores. De seguida, é obtido um código de entrada (*join code*) único através do `RelayService.Instance.GetJoinCodeAsync()`, que pode ser partilhado com outros jogadores. Depois, os dados necessários da alocação (IP, porta, chave e dados da conexão) são configurados no transporte (através do método `SetHostRelayData()`) do `NetworkManager`, e o *host* é iniciado com `NetworkManager.Singleton.StartHost()`. O método retorna o *join code*, este é posteriormente publicado no *lobby* pelo *host*, através do campo personalizado `StartGame_Code`, permitindo que os restantes jogadores o detetem.

Cada jogador, ao verificar que o `StartGame_Code` deixou de ser `"0"`, interpreta isso como sinal de que o jogo vai começar. Através desse código, os clientes ligam-se à sessão *Relay* criada pelo *host*, para tal utilizam o método `JoinRelay(string joinCode)` do `RelayManager`. Este método chama `RelayService.Instance.JoinAllocationAsync(joinCode)` para obter os dados da sessão e configurar o *UnityTransport* com `SetClientRelayData()`, iniciando depois o cliente com `NetworkManager.Singleton.StartClient()`.

Com a utilização dos métodos referidos anteriormente, o tráfego de jogo como movimentação de personagens, ações e eventos passam a ser transmitidos via *Relay*, garantindo a conectividade e sincronização entre todos os jogadores da sessão.

4.2 Mapa

Os mapas para os níveis de jogo foram desenvolvidos com base na estética *Low-Poly* definida no GDD (*Game Design Document*) e os *designs* das várias salas das *dungeons*, que foram desenhados pelo grupo durante uma breve fase de conceção de ideias antes da produção dos mapas. Estes *designs* encontram-se presentes no Apêndice A.

O mundo do jogo **Again** é uma *dungeon* composta por uma zona de descanso chamada **Nexus** e dez andares onde os jogadores têm de enfrentar inimigos com o objetivo de avançar para andares cada vez mais profundos e difíceis com o objetivo de derrotar os *Bosses* presentes no quinto e décimo andar.



Figura 4.2: Zona do Nexus presente no Menu principal.

O *Nexus* é constituído por zonas onde os jogadores podem melhorar as suas personagens, ganhando assim mais pontos de vida e novas habilidades, o que é essencial para progredir no jogo.

Para representar os diferentes andares foram criadas oito tipos de salas diferentes e a partir de diferentes organizações das mesmas foram criados quatro andares diferentes, dois para andares normais e outros dois para andares com *Bosses*.

De maneira a cada andar não ter sempre o mesmo mapa, o `GameManager` escolhe aleatoriamente qual será o próximo mapa ao verificar primeiro se todos os jogadores estão prontos para avançarem para o próximo nível, depois verifica em que andar os jogadores se encontram e gera um número aleatório entre zero e um, se o andar atual estiver entre os intervalos de um a quatro ou seis a nove o número aleatório escolherá entre os mapas *Dungeon1 e Dungeon2*. Caso estejamos no quinto ou décimo andar o número aleatório escolherá entre os mapas *Dungeon3 e Dungeon4*.

Para produzir as diferentes salas foram utilizados *Assets* de estilo *Low Poly* disponíveis na *Unity Asset Store*. Posteriormente foram aplicados componentes como *Box Colliders* aos *Assets* de forma a impedir os jogadores de atravessarem estruturas e outros objetos.

Algumas salas contêm **Hazards** (perigos do ambiente) que dificultam a *gameplay* dos jogadores, obrigando-os a adaptar a sua movimentação. Os principais perigos implementados são as áreas de lava e as serras giratórias.

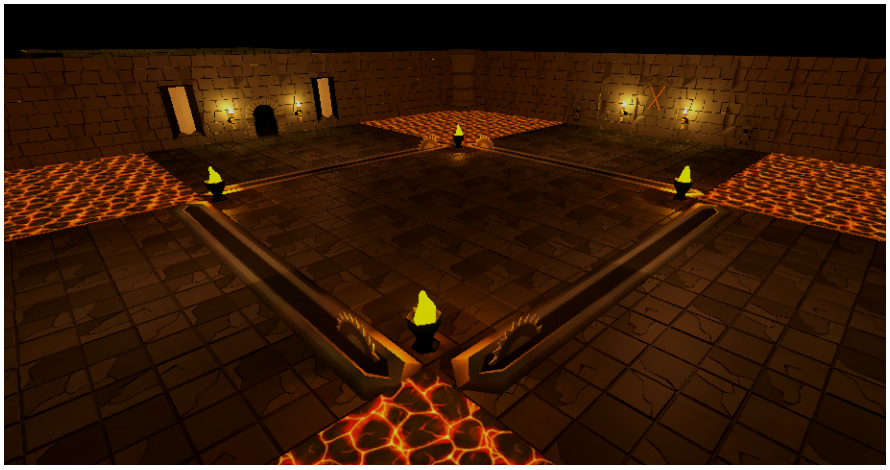


Figura 4.3: Sala com os perigos do ambiente.

A **lava** é uma superfície presente em algumas salas que causa dano contínuo ao jogador enquanto este permanecer em contacto com ela. A deteção de colisão com a lava é feita através do método `OnCollisionEnter`, onde verifica-se se o objeto com que o jogador colidiu possui a tag `"Lava"`. Quando esta condição verifica-se, inicia-se uma corrotina chamada `LavaDamageCoroutine()`, responsável por infligir dano ao longo do tempo. Esta corrotina aplica um dano de três unidades por segundo, chamando repetidamente a função `TakeDamageServerRpc(3f)` enquanto o jogador estiver em contacto com a lava. Assim que o jogador sai da lava, detetado no método `OnCollisionExit`, a corrotina é terminada, interrompendo assim o dano contínuo.

Já as **serras giratórias** funcionam como obstáculos móveis que causam dano imediato ao contacto. Quando o jogador colide com uma serra, o evento `OnCollisionEnter` deteta a tag `"Saw"` e invoca diretamente a função `TakeDamageServerRpc(20f)`, infligindo vinte pontos de dano de forma instantânea. Ao contrário da lava, o dano das serras não é contínuo nem baseado em permanência, mas sim aplicado no momento do impacto, tornando-as particularmente letais quando combinadas com o movimento dos jogadores ou com outros inimigos presentes na sala.

Estas duas mecânicas introduzem variabilidade e desafio adicional ao nível de design, criando situações em que os jogadores precisam de equilibrar combate, navegação do espaço e gestão de vida. São exemplos concretos

de como o ambiente do jogo também pode funcionar como fonte de ameaça ativa, contribuindo para uma jogabilidade mais estratégica.

Por fim, cada sala contém em si uma câmera própria que os jogadores podem acessar quando as suas personagens morrem de modo a, mesmo mortos, poderem assistir ao que está a acontecer com os outros jogadores.

4.3 Personagens e Classes

No projeto desenvolvido, foram concebidas três personagens jogáveis diferentes, cada uma com uma classe própria, com habilidades distintas das restantes, em que se enquadra.

De forma a evitar duplicação de código e a garantir uma estrutura modular, foi criada a classe `CharacterBase`, que serve de classe base para as três personagens jogáveis: Guerreiro, Mago e Clérigo. Esta classe contém toda a lógica comum a todas as personagens, abstraindo funcionalidades como movimentação, rotação, gestão de vida, interação com o sistema de rede e UI, *cooldowns* de habilidades e lógica de morte e renascimento.

A movimentação e orientação da personagem estão centralizadas no método `MovementAndAnimation()`, que aplica deslocamento físico usando o componente `Rigidbody` e ajusta os parâmetros da animação de acordo com a direção do movimento. No caso de dispositivos móveis, é usada a função `RotateWithRightStick()`, enquanto nos restantes casos a orientação é baseada na posição do cursor do rato, usando o método `RotateToMouse()`, que realiza um *raycast* a partir da câmara para determinar a direção de rotação.

A lógica de vida é gerida por variáveis de rede (`NetworkVariables`) como `currentHealth`, `maxHealth`, e `isAlive`, permitindo sincronizar o estado da personagem entre o *host* e os clientes. Quando a vida atinge zero, a função `HandleDeath()` é invocada, ativando a animação de morte, desativando os controlos do jogador e iniciando o processo de reposicionamento ou *respawn*.

No que diz respeito à interação com a interface do utilizador, o método `FindAndSetupUI()` associa a personagem ao HUD correspondente e atualiza elementos visuais como a barra de vida, quantidade de moedas e efeitos ativos como o *Buff* do Clérigo. A integração com o sistema de *saves* também é realizada aqui, através do carregamento dos dados do utilizador e da invocação de chamadas remotas para restaurar o progresso usando os métodos

`UpdateUserServerRpc()` e `LoadPlayerServerRpc()`.

A classe base encapsula a lógica de *cooldowns* das habilidades através de *corrotinas* específicas para cada habilidade (`Hability2CooldownRoutine()`, `Hability3CooldownRoutine()`, `Hability4CooldownRoutine()`), que desativam temporariamente as habilidades, atualizando o estado visual dos botões na UI e reativando-os após um tempo definido.

Quanto à lógica de dano, esta pode ser causada por ataques inimigos ou armadilhas no ambiente. O método `TakeDamageServerRpc()` reduz a vida atual e ativa efeitos visuais, enquanto a função `OnHealthChanged()` sincroniza a informação com a UI e verifica se o jogador morreu. O código também contém uma lógica para bloqueio de ataques, *buffs* temporários (`ActivateBuffServerRpc()`) e interações com objetos do mapa como *triggers* de níveis, inimigos e *spawns*.

Com esta arquitetura orientada a herança, cada classe específica (como `Mage`, `Knight` e `Cleric`) pode sobrepor e estender comportamentos da `CharacterBase`, definindo as suas próprias habilidades sem duplicar a lógica comum, o que facilita a expansão e reuso do código.

4.3.1 Knight

Uma das classes jogáveis é o **Knight** que representa uma personagem do tipo guerreiro, com foco em combate corpo a corpo. Esta personagem é ideal para os jogadores que têm como objetivo fazer parte de um combate mais próximo aos inimigos, com habilidades como bloqueios, ataques em área e um projétil que causa muito dano mas é lento. A sua implementação estende da classe base `CharacterBase`, herda funcionalidades comuns, mas introduz comportamentos específicos, nomeadamente ao nível das habilidades desbloqueáveis a partir do segundo nível da personagem.



Figura 4.4: Classe jogável Knight.

A primeira habilidade é o bloqueio de ataques (`Block`), disponível a partir do nível 2. Esta habilidade permite ao jogador reduzir o dano recebido, ao ativar uma animação de defesa enquanto pressionar um botão de controlo (diferenciado entre PC e dispositivos móveis). A lógica está implementada nos métodos `OnBlock`, `OnBlockMobileStart` e `OnBlockMobileEnd`, que alternam a variável `blocking` e atualizam o estado da animação `"isBlocking"` no `Animator`. Se o jogador bloquear com sucesso múltiplos ataques (mais de cinco hits), o bloqueio é automaticamente cancelado, acionando a corrotina de `cooldown`.

A segunda habilidade corresponde a um ataque giratório em área. Ao ser ativada, define-se a variável `spin = true`, o que bloqueia a movimentação normal da personagem temporariamente e ativa a animação correspondente. Durante a execução da habilidade, o método `KnightAttck3ServerRPC()` é invocado, criando uma esfera de colisão em torno da personagem com um raio de três unidades. Todos os inimigos dentro deste raio são detetados e recebem dano, sendo este amplificado caso o jogador esteja sob o efeito de um *buff*. O efeito visual associado é um `ParticleSystem`, ativado no método `EnableAttack3()`.

A terceira habilidade, considerada a **Ultimate**, é um projétil em linha reta. A sua ativação define a variável `ultimate = true` e executa a animação da habilidade final. No evento de animação, é invocada a função remota

`KnightUltimateServerRPC()`, que instancia um *prefab* associado à habilidade na posição e direção da personagem. Esse objeto é sincronizado em rede através do componente `NetworkObject` e possui um *script* próprio para movimentação e cálculo de dano. Tal como na habilidade anterior, o dano da `Ultimate` pode ser multiplicado por um fator de dois em caso de *buff* ativo.

A classe `Knight` gere ainda o estado de movimentação durante as habilidades, impedindo o jogador de se mover enquanto está a girar ou a usar a *Ultimate*. Esta lógica está centralizada no método `FixedUpdate()`, que anula o vetor de movimento (`_direction`) sempre que uma dessas ações está ativa.

A gestão de partículas está aplicada com métodos como `EnableAttack1Slash()` e `DisableAttack1Slash()` para ativar ou desativar efeitos visuais associados a cada ataque. Estes efeitos são sincronizados entre os jogadores com chamadas `ServerRpc` que comunicam o estado de ataque.

A classe **Knight** é uma personagem versátil, capaz de alternar entre defesa e ataque, com um leque de habilidades que a tornam adequada tanto para absorver dano como para eliminar múltiplos inimigos.

4.3.2 Mage

Outra classe jogável é o **Mage**, que representa uma personagem especializada em ataques de longo alcance e em causar dano em área. Esta classe é ideal para jogadores que preferem manter distância dos inimigos e atacá-los com projéteis e explosões. A sua implementação herda da classe base `CharacterBase`, reutiliza funcionalidades comuns mas acrescenta comportamentos específicos, como a gestão de feitiços, áreas de efeito e projéteis.



Figura 4.5: Classe jogável Mage.

A primeira habilidade, desbloqueada a partir do nível 2, é um ataque com uma bola de fogo que explode ao atingir um inimigo. A sua ativação ocorre através dos métodos `OnAttack2` e `OnAttack2Mobile`, que configuram a animação (`"Habilidade1"`) e acionam a corrotina de *cooldown*. No evento de animação, o método `MageSkill1ServerRPC()` é executado, instanciando o projétil de fogo (`_attck2`). Este projétil é sincronizado em rede e causa dano amplificado em caso de *buff*, além de possuir características específicas de explosão (`SetFireBall`).

A segunda habilidade, disponível a partir do nível 3, é um meteoro de gelo numa área escolhida. No PC, a posição é definida com base na posição do rato (`_worldMousePosition`), obtida por `Raycast` no ambiente 3D. Em dispositivos móveis, é calculada uma posição padrão à frente da personagem. O método `MageSkill2ServerRPC()` é então chamado, instanciando o efeito (`_attck3`) na posição definida. Todos os inimigos dentro de um raio de seis unidades são detetados, e o dano é calculado com base na distância ao centro do impacto, garantindo que inimigos mais próximos recebem mais dano, com um valor mínimo de quarenta.

A terceira habilidade é a **Ultimate**, desbloqueada no nível 4 e representa uma explosão ao redor da personagem. Quando ativada, executa uma animação (`"Habilidade3"`) e bloqueia o movimento do jogador enquanto

estiver ativa. O método `MageUltimateServerRPC()` é chamado durante a animação e deteta todos os inimigos num raio de três unidades em torno da personagem e aplica dano direto a cada um deles. O dano pode também ser amplificado por um *buff* ativo. O efeito visual é gerido pelo `ParticleSystem _ultimatePrefab`, que é ativado no método `EnableUltimate()`.

Tal como no `Knight`, o `Mage` bloqueia o movimento do jogador durante a execução das habilidades especiais, garantindo que a personagem não se move enquanto canaliza um feitiço. Esta lógica é tratada no método `FixedUpdate()`, que anula o vetor de movimento (`_direction`) sempre que as variáveis `skill` ou `ultimate` estão ativas.

A classe **Mage** é uma personagem focada em causar grande dano, com habilidades focadas a dar dano em grupos de inimigos. O seu estilo incentiva uma jogabilidade à distância, equilibrando o posicionamento e a utilização inteligente de *cooldowns* para maximizar o dano em combate.

4.3.3 Cleric

A classe **Cleric** é uma personagem de suporte, focada em curar aliados, aplicar *buffs* e até reviver membros da equipa. Esta classe é essencial para jogadores que pretendem assumir um papel de apoio, garantindo a sobrevivência do grupo durante combates mais difíceis. Tal como as restantes classes, herda da `CharacterBase` e introduz comportamentos específicos para habilidades de cura, *buff* e ressurreição.



Figura 4.6: Classe jogável Cleric.

A primeira habilidade, desbloqueada no nível 2, é uma cura em área. A ativação ocorre pelos métodos `Healing` e `HealingMobile`, que definem o *trigger* da animação `"Healing"` e impedem o movimento da personagem através da variável `_noMove`. No evento de animação, o método `HealOrBuff(1)` é chamado, o que ativa o `ParticleSystem heal` e invoca a função remota `AreaHealServerRpc()`, que aplica cura a todos os aliados dentro de um raio de quatro unidades.

A segunda habilidade, desbloqueada no nível 3, é um *buff* em área. Funciona de forma semelhante à cura: é ativada pelos métodos `Buff` e `BuffMobile`, que definem a animação `"Buff"` e bloqueiam o movimento da personagem. O evento de animação chama `HealOrBuff(2)`, que ativa o `ParticleSystem buff` e executa `AreaBuffServerRpc()`, aplicando o *buff* a todos os aliados próximos.

A terceira habilidade é a **Ultimate**, desbloqueada a partir do nível 4, que permite ao `Cleric` reviver todos os jogadores caídos. A ativação é feita através dos métodos `Revive` e `ReviveMobile`, que bloqueiam o movimento e acionam a animação `"Habilidade4"`. Durante a execução, o método `ReviveParticleSystem()` é chamado, e ativa o `ParticleSystem revivePs` e a função `ReviveAllPlayersServerRpc()`, que percorre todos os jogadores e define o estado de revive nos que estiverem mortos.

O **Cleric** é uma classe focada para estratégias cooperativas, mantendo

a equipa viva com curas, fortalecendo aliados com *buffs* e garantindo uma segunda oportunidade através da ressurreição em grupo.

4.4 Inimigos

No projeto, os inimigos são elementos que interagem com os jogadores, utilizando uma lógica baseada em máquinas de estados para gerir comportamentos como patrulha, perseguição e ataque. Estes inimigos são instanciados dinamicamente pela classe *Spawner* em posições aleatórias dentro de uma área limitada (cada inimigo só desloca-se na sua respetiva sala) e adaptam as suas características (vida, dano e recompensas) consoante o nível do jogo e o número de jogadores.

Em cada nível, para se progredir no jogo, os jogadores têm de matar no mínimo 8 inimigos ou caso o nível tenha um *Boss*, é necessário eliminá-lo para conseguir-se passar para o próximo mapa.

A classe *Spawner* é responsável por criar os inimigos no mapa. Quando o primeiro jogador entra numa nova sala, chama o método `RequestSpawnEnemiesServerRpc()`, que calcula a quantidade de inimigos a gerar com base no número de jogadores na sessão e no nível atual. Este código também pode instanciar um inimigo especial do tipo *Boss*, sendo que isto só se realiza nos níveis 5 e 10.

Para cada inimigo gerado na sala, são definidos os valores de vida, dano e moedas (quanto dá aos jogadores ao morrer). Estes valores aumentam conforme o nível e o tipo de inimigo, para assim criar um maior desafio ao longo da sessão de jogo. Os inimigos são posicionados aleatoriamente dentro dos limites da sala, utilizando o `NavMeshAgent` para navegação.

Todos os inimigos compartilham uma máquina de estados baseada no sistema de animação do `Unity (Animator com StateMachineBehaviour)`, que controla as suas ações. A estrutura comum inclui:

- `Walk (WalkSkeleton)`: O inimigo move-se aleatoriamente dentro da sala, definindo novos destinos quando atinge o ponto atual.

- `Search (SearchSkeleton)`: O inimigo pára no ponto e realiza a animação de Idle quando esta termina volta-se a deslocar para outro ponto

`(navMeshAgent.isStopped = true/false`, no início e fim da animação).

- `Run (ChaseSkeleton)`: Se um jogador é detetado (através de um *Raycast* no método `Saw`), ou se ele estiver a uma certa distância do jogador (menos de 12 de distância) o inimigo persegue-o, atualizando constantemente o seu destino.
- `Ataque (AttackSkeleton)`: O inimigo rotaciona suavemente em direção ao jogador e executa a animação de ataque.
- `Morte`: Quando a vida chega a zero, o inimigo executa uma animação de morte e é removido após um breve intervalo de tempo (cinco segundos).

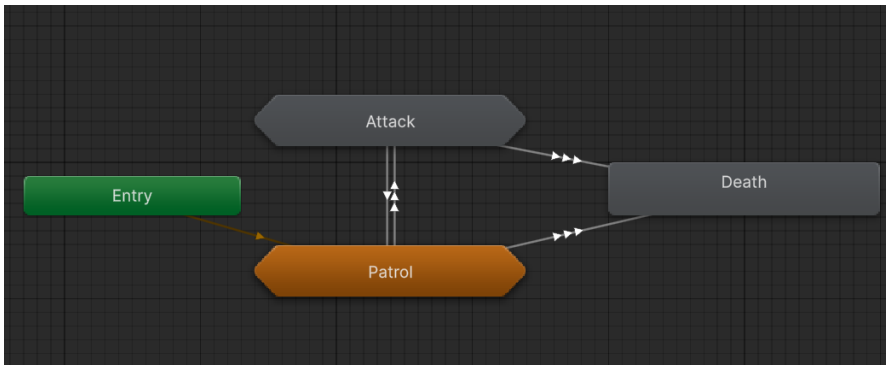


Figura 4.7: Animator dos inimigos.

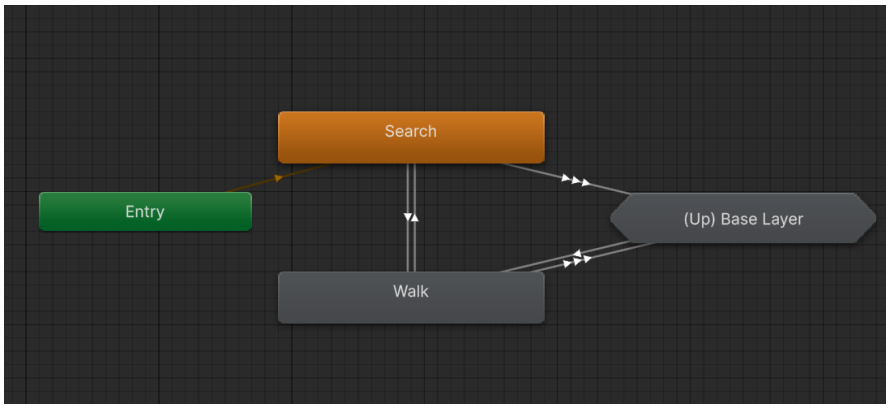


Figura 4.8: Sub-State Machine Patrol do inimigo.

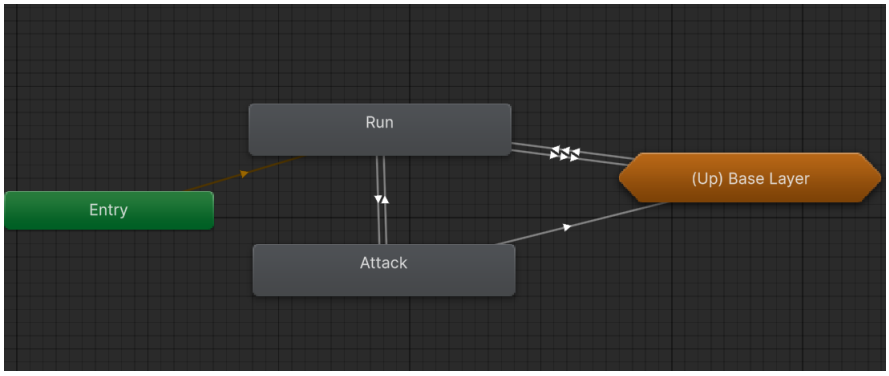


Figura 4.9: Sub-State Machine Attack do inimigo.

Os inimigos no jogo distinguem-se não apenas pelos seus atributos, mas também pelos seus comportamentos de ataque, animações e mecânicas de combate. Todos os inimigos possuem armas visuais nas mãos, cada uma equipada com um *Box Collider* marcado com a tag **"EnemyWeapon"**. Quando estas armas entram em contacto com os jogadores, é acionado o sistema de dano.

O dano é processado através do método `OnTriggerEnter()` nos jogadores, que deteta colisões com as armas dos inimigos. Quando um jogador é atingido, é chamado o método `DamageFromSkeletonServerRpc()`. O dano efetivo é obtido através do método `GetDamage()` da classe `Skeleton`, que só retorna um valor superior a zero se o inimigo estiver no estado de ataque (`isAttacking = true`). Este `bool` é controlado por eventos de animação, garantindo que o dano só é aplicado durante os *frames* relevantes nas animações de ataque.

O comportamento dos inimigos é implementado na classe `Skeleton`, que define a base de movimentação, deteção de jogadores, cálculo de trajetos válidos com `NavMeshPath`, gestão de vida, dano, ataques (simples e com possibilidade de combo), além de funções para destruição e distribuição de recompensas. O sistema de deteção de jogadores verifica constantemente a posição de todos os jogadores ativos, procurando o mais próximo que esteja dentro da área da sala e com um trajeto alcançável. No projeto foram desenvolvidos dois tipos de esqueletos:

- `Inimigo Base (combo = false)`: Esqueleto com uma espada pequena, ele executa uma única animação de ataque, definindo `isAttacking`

como `true` durante o movimento ofensivo. Não possui *combos*, tornando-o previsível, mas ainda assim perigoso em grupos.

- `Inimigo com Long Sword (combo = true)`: Esqueleto com uma espada maior e com um escudo nas costas. Ele possui um ataque básico e tem 50 por cento de chance de executar um *combo* (controlado pelo parâmetro `"Combo"no Animator`), se *combo* for `true` ele executa outra animação de ataque. O método `ResetAttack()` da classe `Skeleton` determina aleatoriamente se o *combo* é ativado, adicionando variedade ao seu comportamento.



Figura 4.10: Inimigos base - Esqueletos.

O **Boss** como possui atributos diferentes dos outros inimigos, mas possui todas as suas funcionalidades ele herda da classe `Skeleton`, ele introduz uma mecânica de duas fases:

- Fase 1: Utiliza uma única animação de ataque, semelhante ao inimigo Base.
- Fase 2 (Ativada ao atingir 50 por cento de vida): Desbloqueia duas animações de ataque adicionais, selecionadas aleatoriamente pelo parâmetro `"Atck"` no *Animator* (definido no método `OnStateExit` do

estado `ChaseSkeleton`) se o valor de `Atck` for menor de 0.5 ele realiza uma animação de ataque, se não executa a outra. Além disso, durante esta segunda fase, o dano do *Boss* é multiplicado por 1.5x. A transição entre fases é feita no método `ReceiveDamage()`, e todo o comportamento do *Boss* está encapsulado na sua própria classe, que substitui e expande métodos como `GetDamage()`, `SetHp()` e `DestroyAfterTime()` para lidar com esta lógica adicional.



Figura 4.11: Boss Minotauro.

4.5 SinglePlayer

No projeto, o grupo optou por desenvolver uma versão *singleplayer* do jogo. Para evitar a duplicação do projeto e a criação de uma versão completamente separada e *offline* do jogo, optou-se por uma abordagem mais prática e eficiente, o modo *singleplayer* foi implementado reutilizando a mesma base de código *multiplayer*, desenvolvida com o *Netcode for GameObjects*. Em vez de remover todo o sistema de rede ou criar uma lógica separada, o jogo inicia uma sessão como *host* (jogador que é simultaneamente servidor e cliente), mas restringe o acesso ao *lobby* para que apenas um jogador possa participar.

Quando o jogador inicia o jogo em modo *singleplayer*, é criado automaticamente um *lobby* privado, sem exibir o código de acesso a outros jogadores.

Esta omissão do código de *join* impede qualquer tentativa de entrada externa, garantindo que a sessão decorra como uma experiência completamente isolada. Do ponto de vista da rede, o jogo continua a funcionar com a mesma lógica, sincronizando objetos, estados e ações localmente, como se estivesse numa sessão multijogador, mas com apenas um jogador ativo.

Esta solução traz várias vantagens:

- Reutilização do código: Não é necessário manter duas versões distintas do jogo (uma *online* e outra *offline*), o que reduz significativamente a complexidade do projeto.
- Coerência dos comportamentos: Toda a lógica de *gameplay*, como *spawn* de inimigos, deteção de jogadores e distribuição de recompensas, continua a funcionar da mesma forma que no modo multijogador, evitando discrepâncias entre os modos.
- Transição facilitada: É possível adaptar rapidamente funcionalidades do modo *multiplayer* para o *singleplayer* (e vice-versa), pois ambos compartilham a mesma base técnica.

Contudo, esta abordagem também apresenta algumas desvantagens, como a lógica do jogo fica fortemente acoplada ao sistema de rede, ou seja, métodos simples como "dar dano"ou "destruir objeto"exigem chamadas via `ServerRpc`, quando poderiam ser funções locais. Isso torna o código mais verboso e menos intuitivo para quem espera uma lógica direta para um modo *singleplayer*.

No contexto do desenvolvimento com *Netcode for GameObjects*, esta abordagem é relativamente comum. Muitos jogos implementam o modo *singleplayer* desta forma por conveniência, especialmente quando o jogo já é fortemente baseado em uma lógica de rede. Evita-se assim duplicação de lógica, facilitando a manutenção e o desenvolvimento contínuo.

4.6 UI

A interface do utilizador do jogo foi desenvolvida com foco na clareza, na adaptabilidade a diferentes plataformas (*desktop* e *mobile*) e na interação eficiente com os sistemas centrais do jogo. O *script* principal responsável por

gerir os elementos da UI durante o jogo é o `GameUI.cs`. Este script centraliza todos os componentes visuais e as interações diretas do jogador com o HUD (vida, habilidades, botões de ação, entre outros).

4.6.1 Visualização de Vida e Estado do Jogador

O sistema de barras de vida utiliza dois `Slider`: um para a vida atual (`healthSlider`) e outro para um efeito de transição suave (`easeHealthSlider`). A função `Update()` estima o valor intermédio entre os valores, criando um efeito visual fluido ao receber dano. Este sistema é complementado com métodos auxiliares como `TakeDamage()` e `Heal()`.

4.6.2 Controlo de Habilidades e Plataforma

A UI adapta-se dinamicamente à plataforma: em dispositivos móveis são ativados os botões de ataque (`basicAttackButton`, `ability1Button`, etc.), bem como os *joysticks* virtuais (`mobileMoveStick` e `mobileAimStick`). O método `SetPlayer()` identifica a classe do jogador (`Knight`, `Mage` ou `Cleric`) e configura os botões correspondentes com *listeners* apropriados. Por exemplo, no caso do `Knight`, o botão `ability1` é configurado com eventos `PointerDown` e `PointerUp` para ativar e desativar a defesa.

4.6.3 Atualização de Imagens e Progresso de Habilidades

A aparência dos ícones das habilidades muda com o progresso do jogador. O método `UpdateImages()` altera os `sprites` dos botões e a sua interatividade de acordo com o nível do jogador. Além disso, a função `setImage()` permite alterar individualmente o estado visual de cada habilidade (cor ou preto e branco), refletindo se estão ativas ou bloqueadas.



Figura 4.12: UI durante o jogo.

4.6.4 Menus e Transições

O `GameUI` também controla os vários menus de jogo: ecrã principal, ecrã de opções, *loading screen*, painel de vitória ou derrota. Funções como `OptionsScreen()`, `VictoryToNexus()` e `GameScreenFromOptions()` fazem a transição entre estes menus, ativando/desativando os objetos correspondentes no *canvas* da UI.

4.6.5 Botão de Guardar Jogo

O botão de guardar jogo (`saveGame`) está disponível apenas para o *host*, conforme a lógica em `OptionsScreen()`. Quando clicado, este botão chama o método `SaveGame()`, que por sua vez invoca a função `SaveGameSession()` do `GameManager`.

A função `SaveGameSession()` recolhe os dados de todos os jogadores atuais na sessão. Para cada jogador, extrai o ID, a quantidade de moedas, o nível e a classe (`Mage`, `Knight` ou `Cleric`). Estes dados são encapsulados num objeto `PlayerSessionData`. Após recolher todos os dados, a função cria um objeto `GameSessionData` com o nome da sessão (obtido via `ManagerUI.Instance.GetLobbyName()`), uma *flag* `multiplayer`, e a lista dos jogadores. Finalmente, este objeto é enviado para o sistema de salvamento (`SaveSystem.UpdateGameSession()`), que persiste os dados local-

mente.

Este sistema garante que toda a informação necessária para restaurar uma sessão futura está disponível, sendo útil tanto em *singleplayer* como em *multiplayer*.

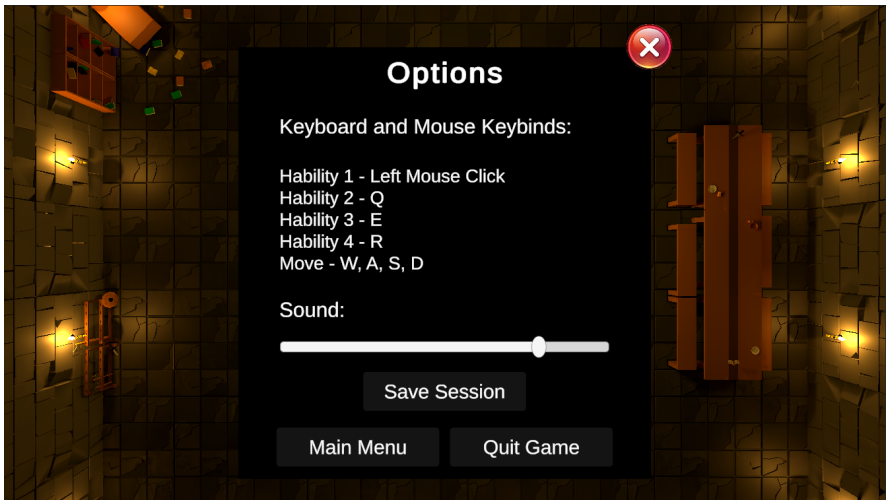


Figura 4.13: UI do menu de opções do jogo.

4.6.6 Interligação com o Menu Inicial

O menu principal é gerido pelo *script* `MainMenu.cs`. Ele determina se o utilizador já existe através de `SaveSystem.LoadUser()` e direciona-o para a escolha do nome de utilizador ou para o menu principal. Já o `ManagerUI.cs` é o controlador central de navegação entre menus e sessões, sendo também responsável por iniciar o `GameUI` com o nome do jogador e gerir o carregamento de sessões guardadas.



Figura 4.14: Menu principal do jogo.

Capítulo 5

Validação e Testes

Após a conclusão do desenvolvimento do jogo, foi realizada uma fase de validação e testes com o objetivo de verificar a estabilidade, desempenho e experiência do utilizador em diferentes dispositivos e condições de rede.

Durante os testes técnicos, o jogo demonstrou um desempenho consistente no ambiente de desenvolvimento. No computador, a taxa de *frames* por segundo (FPS) manteve-se entre 100 e 120 FPS, garantindo uma positiva jogabilidade mesmo com múltiplos elementos ativos no mapa. Em dispositivos Android, a taxa de FPS foi estável em 60 FPS.

No que diz respeito à latência da rede, os testes com múltiplos jogadores em sessões *online* indicaram que os clientes mantinham *pings* na ordem dos 50 ms, um valor considerado adequado para jogos em tempo real, permitindo sincronização entre jogadores e inimigos sem atrasos visíveis.

Para além dos testes anteriores, foi conduzido um processo de validação com utilizadores reais. O grupo elaborou um questionário, presente no Apêndice B, e convidou colegas e familiares a jogarem o jogo. No total, 10 pessoas participaram nos testes, com a seguinte distribuição demográfica:

- Idades: A maioria (6 pessoas) tinha entre 18 e 24 anos, também participaram pessoas entre 16 e 30 anos e uma pessoa com mais de 40 anos.
- Género: 7 masculinos e 3 femininos.
- Frequência de jogo: A maioria joga frequentemente ou diariamente, garantindo um público com alguma familiaridade com videojogos.

- Plataformas testadas: quatro pessoas testaram em ambas as plataformas, cinco testaram apenas no computador e uma apenas no android.

No questionário, os utilizadores avaliaram vários aspetos do jogo, incluindo a jogabilidade, interface, combate e desempenho. De forma geral, o *feedback* foi positivo:

- Todos os utilizadores afirmaram não ter sentido lentidão ou baixa taxa de FPS durante o jogo.
- Os controlos e a interface foram considerados intuitivos, e a latência da rede foi avaliada como baixa.
- O sistema de combate foi considerado fácil de entender e as classes jogáveis foram vistas como interessantes.

Apesar do *feedback* positivo, foram registadas algumas sugestões de melhoria, como o aumento de variedade de inimigos, melhorias na interface gráfica e adição de mais sons.

Os questionários além de avaliarem a experiência do utilizador, também serviram para validar os objetivos principais do projeto. As respostas permitiram confirmar que o jogo alcança uma jogabilidade positiva, funcionalidades completas e uma estrutura compatível com sessões *multiplayer* e *singleplayer*, como inicialmente definido nos objetivos do projeto.

Capítulo 6

Conclusões e Trabalho Futuro

Este projeto teve como objetivo o desenvolvimento de um jogo *Roguelike* multiplataforma (Windows e Android) com funcionalidades *multiplayer*, recorrendo ao motor de jogo Unity3D. Para além de explorar os desafios técnicos associados ao desenvolvimento de jogos com suporte a *multiplayer* e multiplataforma, este trabalho visou também promover uma maior familiarização com metodologias de planeamento, organização e execução de um projeto completo de desenvolvimento de videojogos.

Uma das principais conclusões retiradas é que o desenvolvimento de um jogo com funcionalidades *multiplayer* multiplataforma representa um desafio significativo, sobretudo no que diz respeito à sincronização de estados. A utilização da biblioteca *Netcode for GameObjects*, em conjunto com os serviços de *Lobby* e *Relay* providenciados pelo Unity, permitiu mitigar estes desafios, possibilitando uma melhor experiência entre os jogadores.

O projeto revelou também que, com uma estrutura de trabalho organizada, é possível construir uma base sólida para um jogo completo num espaço de tempo relativamente curto. A implementação de funcionalidades como *lobbies* públicos e privados, classes distintas, inimigos, sistema de combate, bem como um modo *singleplayer* funcional, permitiu alcançar os objetivos do projeto e, ao mesmo tempo, criar um protótipo jogável.

Outro aspeto importante foi a adoção de uma abordagem prática no modo *singleplayer*, onde se recorreu à criação de um *lobby* privado, sem exibição do código de entrada, para simular uma sessão individual utilizando a mesma estrutura base do *multiplayer*. Esta solução mostrou-se eficaz na redução da duplicação de código, embora apresente limitações como a dependência do

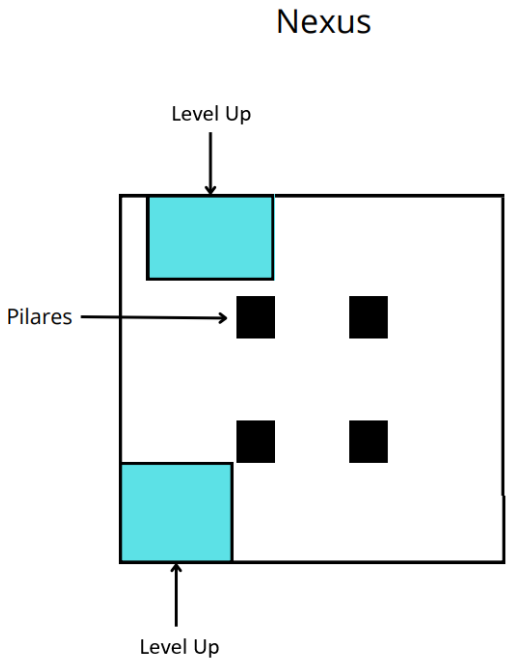
sistema de rede mesmo em sessões *singleplayer*.

Como trabalho futuro, existem várias direções possíveis:

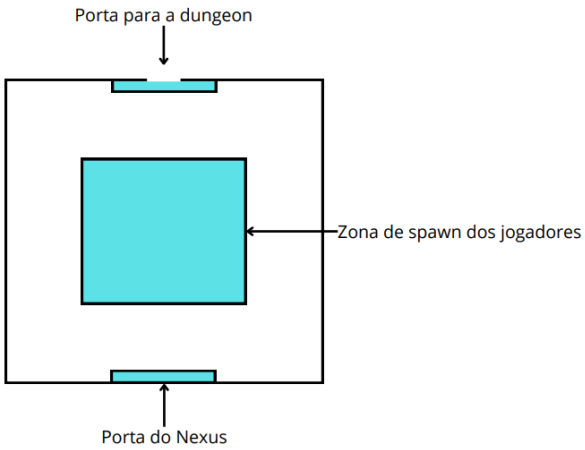
- Expansão do conteúdo jogável, com o desenvolvimento de novas classes, inimigos e habilidades.
- Introdução de geração procedimental de mapas.
- Melhorias na interface gráfica (UI).
- Implementação de um *matchmaking* automático.
- Introdução de NPCs (*Non-Player Character*) que aprofundem a história do mundo e universo de jogo.
- Implementação de um sistema de conquistas, para adicionar novos objetivos para os jogadores.

Apêndice A

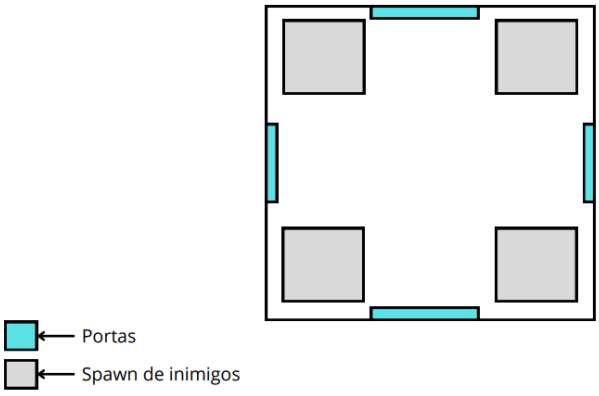
Design das salas



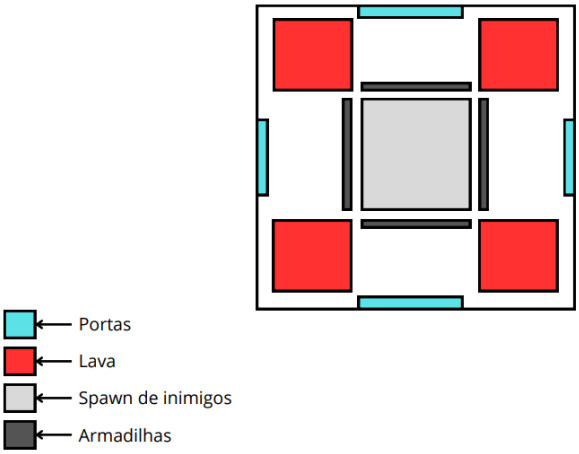
Sala de spawn



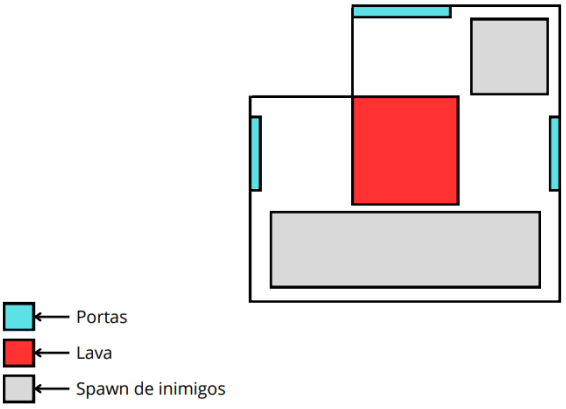
Sala 1



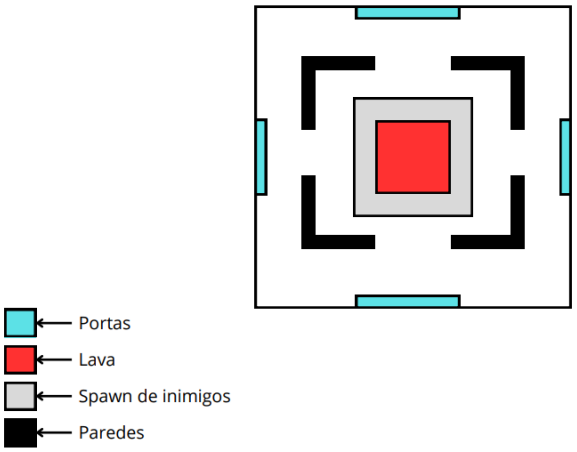
Sala 2


Portas
Lava
Spawn de inimigos
Armadilhas

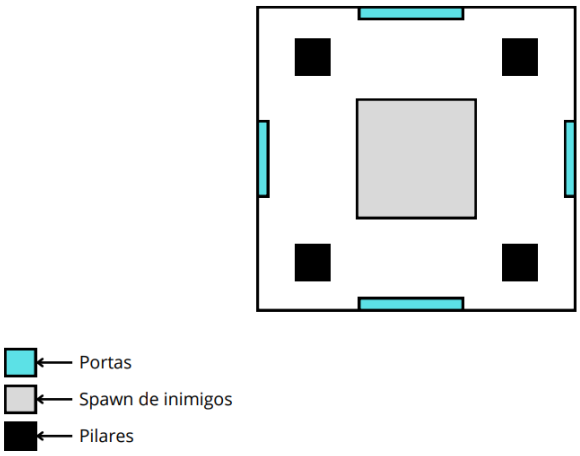
Sala 3


Portas
Lava
Spawn de inimigos

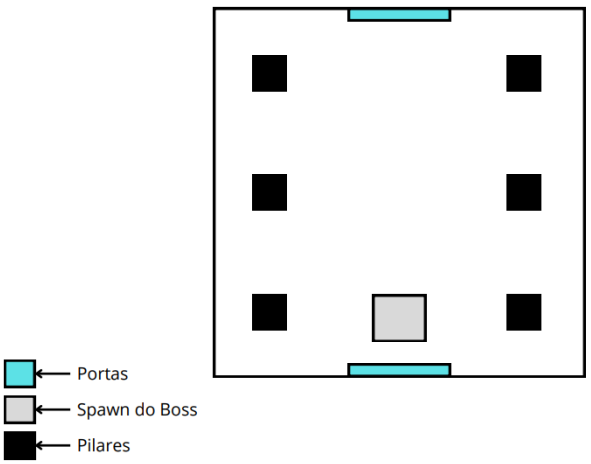
Sala 4


Portas
Lava
Spawn de inimigos
Paredes

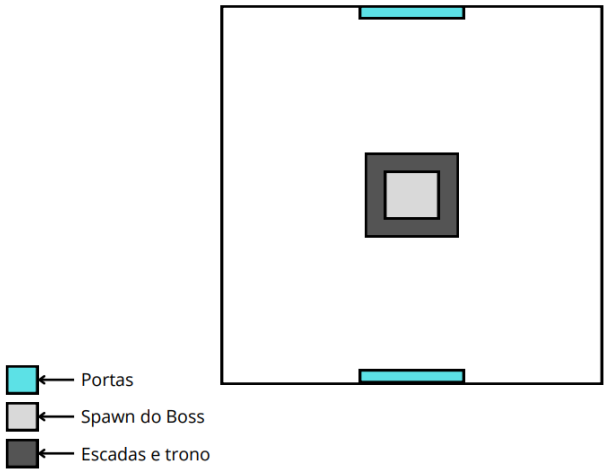
Sala 5


Portas
Spawn de inimigos
Pilares

Boss 1


Portas
Spawn do Boss
Pilares

Boss 2

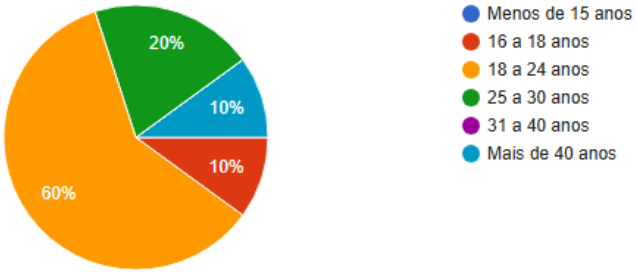

Portas
Spawn do Boss
Escadas e trono

Apêndice B

Questionário de Satisfação

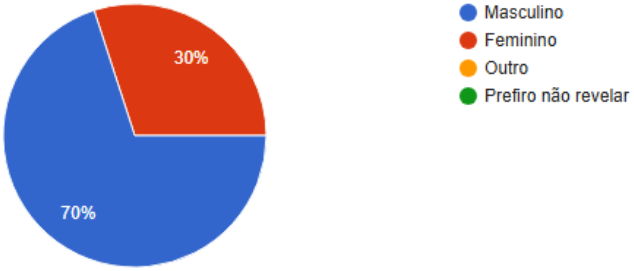
Indique a sua idade.

10 respostas



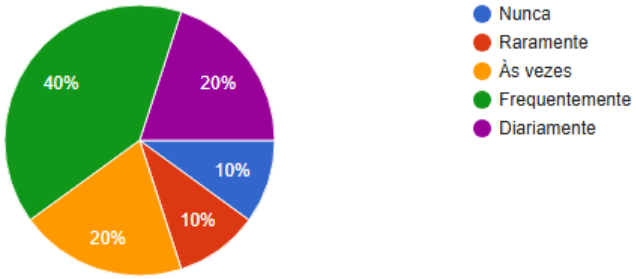
Indique o seu género.

10 respostas



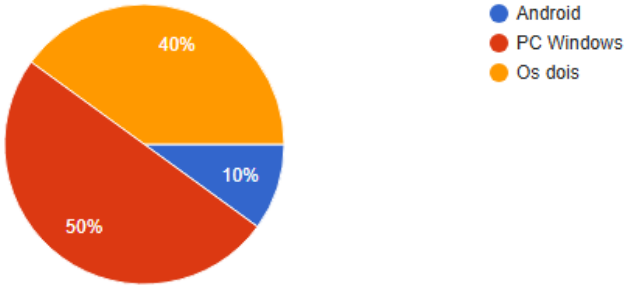
Com que frequência joga jogos?

10 respostas



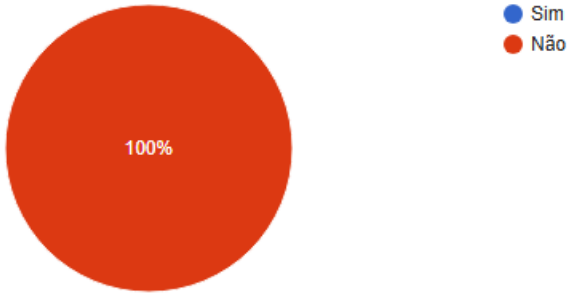
Testou o jogo em que versão?

10 respostas



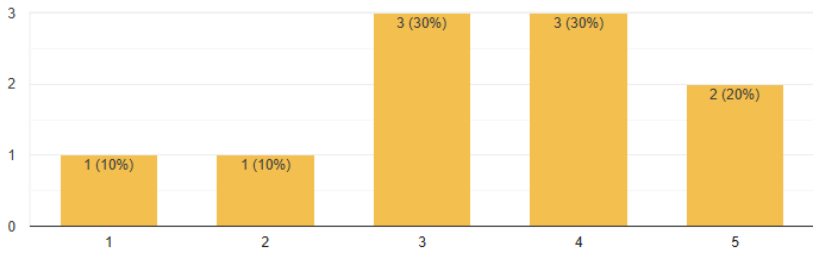
Ao jogar senti o jogo lento (com número baixo de fps)?

10 respostas



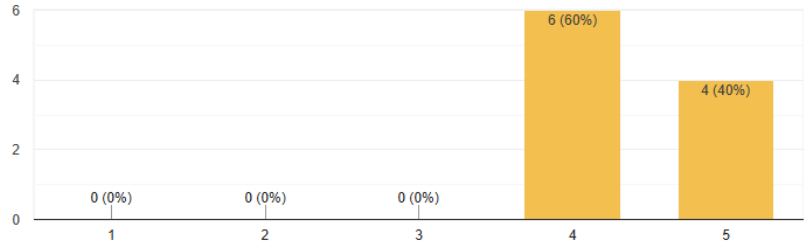
Acho que gostaria de jogar este jogo com frequência.

10 respostas



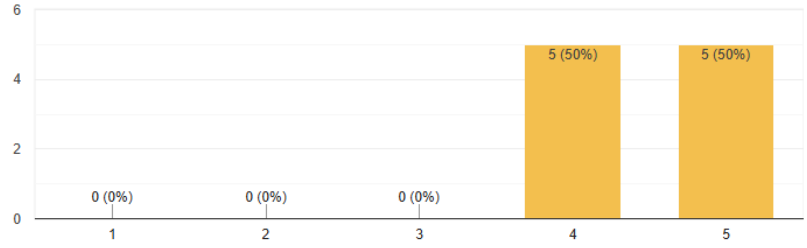
Achei a interface do jogo fácil de utilizar.

10 respostas



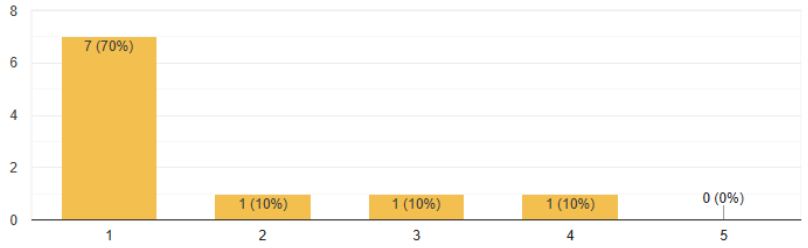
Considerei que as várias funcionalidades deste jogo estavam bem integradas.

10 respostas



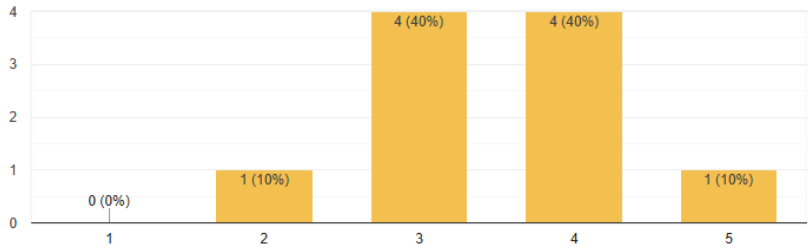
Achei que este jogo tinha muitas inconsistências.

10 respostas



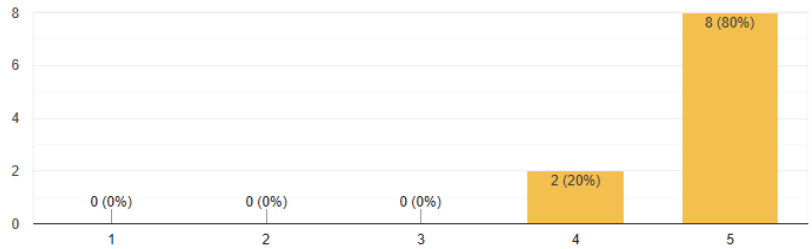
Suponho que a maioria das pessoas consiga jogar com facilidade o jogo.

10 respostas



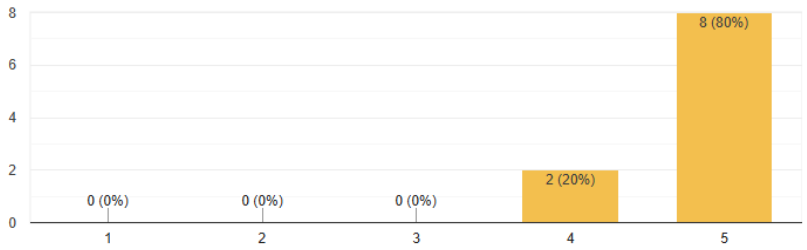
As personagens respondem bem aos comandos.

10 respostas



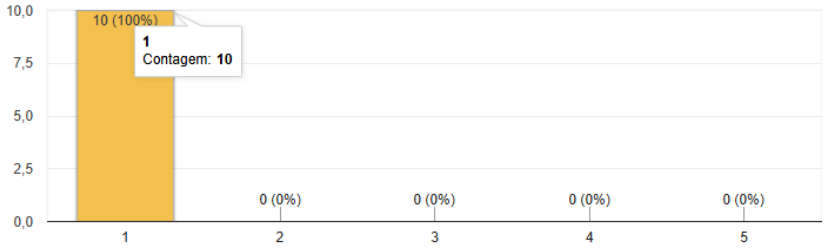
Os controlos são intuitivos e fáceis de aprender.

10 respostas



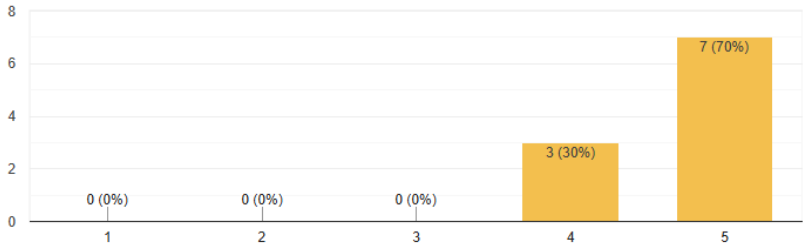
A jogabilidade foi afetada negativamente pelos controlos.

10 respostas



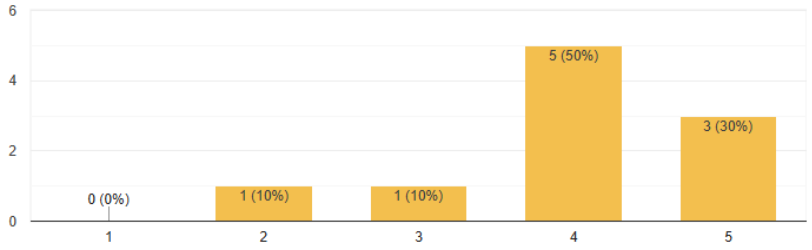
As classes jogáveis têm habilidades distintas e interessantes.

10 respostas



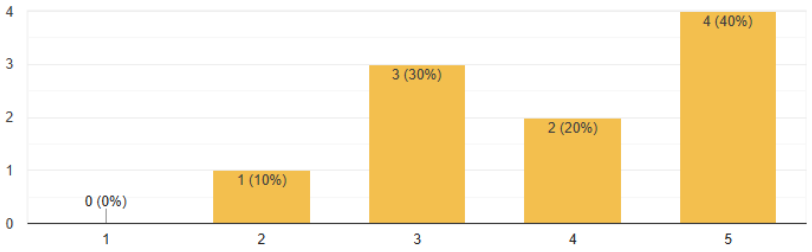
Achei os níveis de jogo esteticamente interessantes.

10 respostas



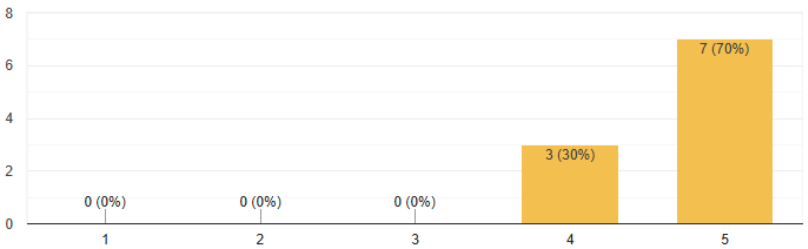
Achei os níveis de jogo interessantes, a nível de jogabilidade.

10 respostas



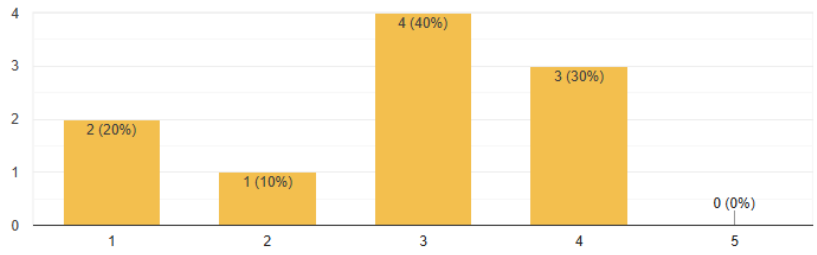
O sistema de combate é fácil de entender.

10 respostas



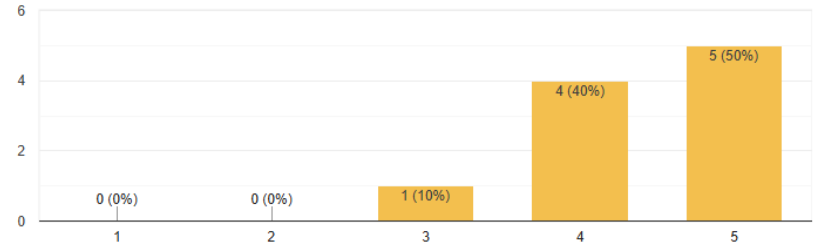
Achei os inimigos interessantes.

10 respostas



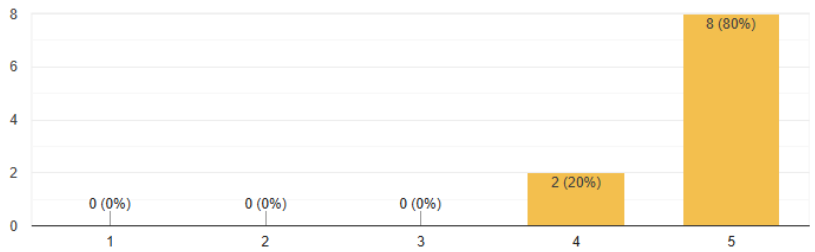
A dificuldade dos inimigos evolui de forma equilibrada ao longo do jogo.

10 respostas



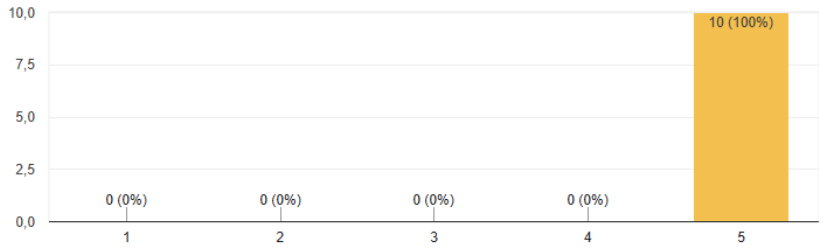
Criar um lobby foi fácil e intuitivo.

10 respostas



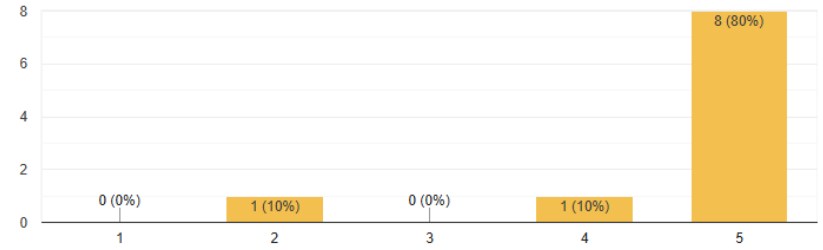
Encontrar um lobby para entrar foi fácil e intuitivo.

10 respostas



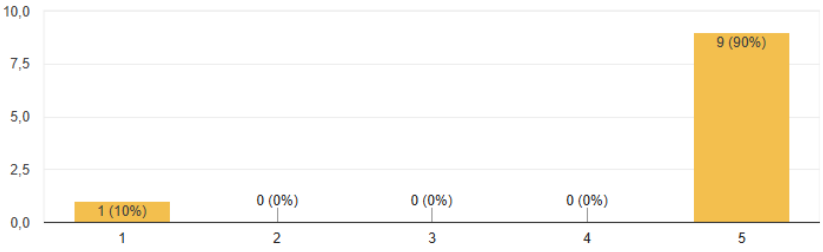
A latência do jogo (atraso entre ações e resposta) foi baixa.

10 respostas



Senti "lag" durante as partidas.

10 respostas



Bibliografia

[ChillyRoom, 2017] ChillyRoom (2017). Soul knight. https://chillyroom.com/en. Accessed June 2025.

[GameScrye, 2024] GameScrye (2024). Game design documents - gamescrye. https://gamescrye.com/resources/game-design-documents/. Accessed June 2025.

[Supergiant, 2020] Supergiant (2020). Hades. https://www.supergiantgames.com/games/hades/. Accessed June 2025.

[Unity Technologies, 2024a] Unity Technologies (2024a). Unity lobby - get started manual. https://docs.unity.com/ugs/en-us/manual/lobby/manual/get-started. Accessed June 2025.

[Unity Technologies, 2024b] Unity Technologies (2024b). Unity multiplayer solutions - landing page. https://docs.unity.com/ugs/en-us/manual/mps-sdk/multiplayer-landing. Accessed June 2025.

[Unity Technologies, 2024c] Unity Technologies (2024c). Unity netcode for gameobjects - documentation. https://docs-multiplayer.unity3d.com/netcode/current/about/. Accessed June 2025.

[Unity Technologies, 2024d] Unity Technologies (2024d). Unity relay - get started manual. https://docs.unity.com/ugs/en-us/manual/relay/manual/get-started. Accessed June 2025.

[Unity Technologies, 2024e] Unity Technologies (2024e). Unity relay and netcode for gameobjects. https://docs.unity.com/ugs/en-us/manual/relay/manual/relay-and-ngo. Accessed June 2025.

[Vlambeer, 2015] Vlambeer (2015). Nuclear throne. https://nuclearthrone.com/. Accessed June 2025.

[Zeekerss, 2023] Zeekerss (2023). Lethal company. https://store.steampowered.com/app/1966720/Lethal_Company/. Accessed June 2025.